

**LAPORAN AKHIR PRAKTIKUM
ALGORITMA & STRUKTUR DATA**



Oleh:

Muhammad Rifqi Rahman

NIM. 2510817310003

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
2026**

LEMBAR PENGESAHAN
LAPORAN AKHIR PRAKTIKUM ALGORITMA & STRUKTUR
DATA

Laporan Praktikum Algoritma & Struktur Data

Modul 1 : Struct dan Pointer

Modul 2 : Stack dan Queue

Modul 3 : Linked List

Modul 4 : Sorting

Modul 5 : Searching

Modul 6 : Graph dan Tree

ini disusun sebagai syarat lulus mata kuliah Praktikum Praktikum Algoritma & Struktur Data. Laporan Akhir Praktikum ini dikerjakan oleh:

Nama Praktikan : Muhammad Rifqi Rahman

NIM : 2510817310003

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhammad Alkaff, S.Kom, M.Kom,
Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	ii
DAFTAR ISI	iii
DAFTAR GAMBAR.....	vii
DAFTAR TABEL	viii
MODUL 1: STRUCT DAN POINTER	1
SOAL 1.....	1
A. Source Code	1
B. Output Program.....	2
C. Pembahasan	2
SOAL 2.....	4
A. Source Code	4
B. Output Program.....	5
C. Pembahasan	5
MODUL 2: STACK DAN QUEUE	7
SOAL 1.....	7
A. Source Code	7
B. Pembahasan.....	8
SOAL 2.....	11
A. Source Code	12
B. Output Program.....	14
C. Pembahasan.....	14
SOAL 3.....	16
A. Source Code	16
B. Pembahasan.....	18
SOAL 4.....	21
A. Source Code	22
B. Output Program.....	24
C. Pembahasan.....	24
MODUL 3: LINKED LIST	2

SOAL 1	2
A. Source Code	2
B. Pembahasan	10
SOAL 2	12
A. Source Code	13
B. Output Program	16
C. Pembahasan	16
SOAL 3	18
A. Source Code	18
B. Pembahasan	26
SOAL 4	28
A. Source Code	29
B. Output Program	35
C. Pembahasan	35
MODUL 4: SORTING	37
KOMPLEKSITAS WAKTU	37
A. Source Code	37
B. Output	47
C. Bubble Sort	48
D. Selection Sort	48
E. Insertion Sort	49
F. Merge Sort	49
G. Quick Sort	49
H. Radix Sort	50
KARAKTERISTIK ALGORITMA	51
A. Bubble Sort	51
B. Selection Sort	51
C. Insertion Sort	51
D. Merge Sort	52
E. Quick Sort	52
F. Radix Sort	52
ANALISIS OPERASI	53

A.	Bubble Sort	53
B.	Selection Sort.....	53
C.	Insertion Sort.....	54
D.	Merge Sort	54
E.	Quick Sort	54
F.	Radix Sort	55
	ANALISIS PERFORMA	56
A.	Perbandingan waktu eksekusi antar algoritma.....	56
B.	Algoritma mana yang paling cepat untuk dataset kecil dan besar	56
C.	Pengaruh distribusi data terhadap performa algoritma	56
D.	Hubungan antara teori kompleksitas dan hasil eksperimen yang diperoleh	57
	MODUL 5: SEARCHING	59
	KOMPLEKSITAS WAKTU	59
A.	Source Code	59
B.	Output	61
C.	Linear Search	62
D.	Binary Search.....	62
	KARAKTERISTIK ALGORITMA	64
A.	Linear Search	64
B.	Binary Search.....	64
	ANALISIS OPERASI	65
A.	Linear Search	65
B.	Binary Search.....	65
	ANALISIS PERFORMA	67
A.	Perbandingan waktu pada kondisi yang sama	67
B.	Efisiensi untuk dataset kecil dan besar	67
C.	Pengaruh kondisi data	67
D.	Teori $O(n)$ vs $O(\log n)$ dan hasil eksperimen	68
	MODUL 6: GRAPH DAN TREE	69
	SOAL 1	69
D.	Source Code	71
E.	Output Program.....	73

F. Pembahasan	73
SOAL 2.....	75
D. Source Code	77
E. Output Program.....	78
F. Pembahasan	78
SOAL 3.....	80
C. Source Code	82
D. Output Program.....	84
E. Pembahasan	84
SOAL 4.....	86
A. Source Code	88
B. Output Program.....	90
C. Pembahasan	90
SOAL 5.....	92
A. Source Code	94
B. Output Program.....	98
C. Pembahasan	98
TAUTAN GIT	101

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Jawaban Soal 1	2
Gambar 2. Screenshot Hasil Jawaban Soal 2	5
Gambar 3. Screenshot Hasil Jawaban Soal 2	16
Gambar 4. Screenshot Hasil Jawaban Soal 4	35
Gambar 5. Screenshot Hasil Bubble Sort	47
Gambar 6. Screenshot Hasil Selection Sort.....	47
Gambar 7. Screenshot Hasil Insertion Sort	47
Gambar 8. Screenshot Hasil Merge Sort	47
Gambar 9. Screenshot Hasil Quick Sort.....	48
Gambar 10. Screenshot Hasil Radix Sort	48
Gambar 11. Screenshot Hasil Linear Search.....	61
Gambar 12. Screenshot Hasil Binary Search.....	62
Gambar 13. Screenshot Hasil Jawaban Soal 1	73
Gambar 14. Screenshot Hasil Jawaban Soal 2	78
Gambar 15. Screenshot Hasil Jawaban Soal 3	84
Gambar 16. Screenshot Hasil Jawaban Soal 4	90
Gambar 17. Screenshot Hasil Jawaban Soal 5	98

DAFTAR TABEL

Tabel 1 Source Code prob1.cpp	1
Tabel 2 Source Code prob2.cpp	4
Tabel 3 Source Code single_linked_list.cpp	2
Tabel 4 Source Code prob_a.cpp.....	13
Tabel 5 Source Code double_linked_list.cpp	18
Tabel 6 Source Code prob_b.cpp	29
Tabel 7 Source Code sorting_algorithms.cpp.....	37
Tabel 8 Source Code sorting_algorithms.cpp.....	59
Tabel 9 Source Code prob1.cpp	71
Tabel 10 Source Code prob2.cpp	77
Tabel 11 Source Code prob3.cpp	82
Tabel 12 Source Code prob4.cpp	88
Tabel 13 Source Code prob5.cpp	94

MODUL 1: STRUCT DAN POINTER

SOAL 1

Diberikan sebuah garis l yang direpresentasikan dalam bentuk persamaan umum:

$$ax + by + c = 0$$

dan sebuah titik $P = (x_p, y_p)$. Tentukan posisi titik P terhadap garis l dengan cara mengimplementasikan fungsi `gradient` dan `CheckPointPosition` pada file header yang telah diberikan.

Input	Output
1 0 -3 5 0	Left
1 0 -3 1 7	Right
2 -1 -1 1 1	On Line

A. Source Code

Tabel 1 Source Code `probl.cpp`

```
1  #include <iostream>
2  #include "point.h"
3  #include "line.h"
4
5  int main() {
6      Line l;
7      Point p;
8
9      while (std::cin >> l.a >> l.b >> l.c >> p.x
10 >> p.y) {
11
12          double gradValue = gradient(&l, &p);
13
14          std::cout <<
CheckPointPosition(gradValue) << "\n";
14      }
```

15	
16	return 0;
17	}

B. Output Program

```

C:\Users\LENOVO\Documents
1 0 -3 5 0
Left
1 0 -3 1 7
Right
2 -1 -1 1 1
On Line

```

Gambar 1. Screenshot Hasil Jawaban Soal 1

C. Pembahasan

Pada baris [1], syntax `#include <iostream>` digunakan untuk mengimpor library standar input output C++ agar program bisa menggunakan perintah seperti `std::cin` dan `std::cout`, yang kemudian diikuti pada baris [2] dan [3] di mana syntax `#include "point.h"` serta `#include "line.h"` digunakan untuk menyertakan file header lokal yang berisi definisi struktur data. Baris [5], syntax `int main()` digunakan untuk mendeklarasikan fungsi utama yang akan dieksekusi pertama kali oleh sistem, dilanjutkan pada baris [6] dan [7] di mana syntax `Line l;` dan `Point p;` digunakan untuk mendeklarasikan objek sebagai wadah penyimpanan nilai masukan. Selanjutnya pada baris [9], syntax `while (std::cin >> l.a >> l.b >> l.c >> p.x >> p.y)` digunakan untuk membuat perulangan yang akan membaca kelima nilai masukan secara terus-menerus selama data masih diberikan. Di dalam perulangan tersebut, tepatnya pada baris [10], syntax `double gradValue = gradient(&l, &p);` digunakan untuk memanggil fungsi kalkulasi posisi titik dengan mengirimkan alamat memori variabel (menggunakan referensi `&`) dan menyimpannya, yang langsung disambung pada baris [11] di mana syntax `std::cout << CheckPointPosition(gradValue) << "\n";` digunakan untuk mengevaluasi variabel tersebut dan mencetak hasil posisinya ke layar. Terakhir, pada baris [14], syntax `return 0;` digunakan untuk mengakhiri fungsi utama secara normal dan memberikan sinyal keberhasilan eksekusi kepada sistem operasi.

SOAL 2

Diberikan sebuah lingkaran C dengan pusat (c_x, c_y) dan jari-jari $r > 0$, serta sebuah titik $P = (p_x, p_y)$. Tentukan posisi titik P terhadap lingkaran C dan mengimplementasikan fungsi `distance` dan `CheckpointInCircle` pada file header yang diberikan.

Input	Output
0 0 5 3 4	On Circle
0 0 5 1 1	Inside
0 0 5 4 4	Outside
-2 -2 4 2 -2	On Circle

A. Source Code

Tabel 2 Source Code prob2.cpp

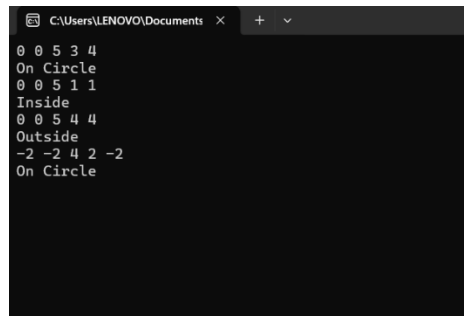
```
1  #include <iostream>
2  #include "point.h"
3  #include "circle.h"
4
5  int main() {
6      Circle c;
7      Point p;
8
9      while (std::cin >> c.centre.x >> c.centre.y
10 >> c.radius >> p.x >> p.y) {
11
12          double dist = distance(&c, &p);
13
14          double diff = dist - c.radius;
```

```

15         std::cout << CheckPointInCircle(diff) <<
16         "\n";
17     }
18     return 0;
19 }

```

B. Output Program



Gambar 2. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Pada baris [1], syntax `#include <iostream>` digunakan untuk mengimpor library standar input output C++ agar program bisa menggunakan perintah seperti `std::cin` dan `std::cout`, yang kemudian diikuti pada baris [2] dan [3] di mana syntax `#include "point.h"` serta `#include "circle.h"` digunakan untuk menyertakan file header lokal yang berisi definisi struktur data lingkaran dan titik. Memasuki baris [5], syntax `int main()` digunakan untuk mendeklarasikan fungsi utama yang akan dieksekusi pertama kali oleh sistem, dilanjutkan pada baris [6] dan [7] di mana syntax `Circle c;` dan `Point p;` digunakan untuk mendeklarasikan objek sebagai wadah penyimpanan nilai masukan. Selanjutnya pada baris [9], syntax `while (std::cin >> c.centre.x >> c.centre.y >> c.radius >> p.x >> p.y)` digunakan untuk membuat perulangan yang akan membaca kelima nilai masukan secara terus-menerus selama data masih diberikan. Di dalam perulangan tersebut, tepatnya pada baris [10], syntax `double dist = distance(&c, &p);` digunakan untuk memanggil fungsi kalkulasi jarak Euclidean antara pusat lingkaran dan titik dengan mengirimkan alamat memori variabel (menggunakan referensi `&`), yang dilanjutkan pada baris [11] dengan syntax `double diff = dist - c.radius;` untuk menghitung selisih antara jarak dan

jari-jari lingkaran. Setelah itu, pada baris [12] syntax `std::cout << CheckPointInCircle(diff) << "\n";` digunakan untuk mengevaluasi nilai selisih tersebut ke dalam fungsi penentu posisi (apakah di dalam, di luar, atau tepat pada lingkaran) dan mencetak hasilnya ke layar. Terakhir, pada baris [15], syntax `return 0;` digunakan untuk mengakhiri fungsi utama secara normal dan memberikan sinyal keberhasilan eksekusi kepada sistem operasi.

MODUL 2: STACK DAN QUEUE

SOAL 1

Pada file `stack.h` yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file `stack.cpp`. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan `throw`.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 1 Source Code stack.cpp

1	<code>#include "stack.h"</code>
2	<code>#include <stdexcept></code>
3	
4	<code>void init(Stack* s) {</code>
5	<code> s->top = s->data - 1;</code>
6	<code>}</code>
7	
8	<code>bool isEmpty(const Stack* s) {</code>
9	<code> return s->top < s->data;</code>
10	<code>}</code>
11	
12	<code>bool isFull(const Stack* s) {</code>
13	<code> return s->top == s->data + MAX - 1;</code>
14	<code>}</code>
15	
16	<code>void push(Stack* s, int value) {</code>
17	<code> if (isFull(s)) {</code>

```

18         throw std::overflow_error("Stack penuh
19         (overflow)");
20     }
21     s->top++;
22     *(s->top) = value;
23 }
24 void pop(Stack* s) {
25     if (isEmpty(s)) {
26         throw std::underflow_error("Stack kosong
27         (underflow)");
28     }
29     s->top--;
30 }
31 int peek(const Stack* s) {
32     if (isEmpty(s)) {
33         throw std::underflow_error("Stack kosong
34         (tidak ada elemen)");
35     }
36     return *(s->top);
37 }

```

B. Pembahasan

Fungsi *init*, *isEmpty*, dan *isFull* beroperasi sebagai fondasi pengelolaan struktur tumpukan dengan memanipulasi alamat pointer secara langsung. Alur fungsi *init* menugaskan pointer *top* ke alamat awal array, memberikan justifikasi yang tepat untuk titik awal tumpukan kosong agar elemen pertama nantinya tersimpan rapi di indeks ke-nol. Fungsi *isEmpty* memeriksa apakah *top* masih berada di alamat awal tersebut, yang secara logis

membuktikan ketiadaan elemen, sementara *isFull* membandingkan posisi *top* dengan batas akhir memori (alamat awal ditambah *MAX*) guna memvalidasi ketiadaan sisa ruang secara matematis. Karena ketiga fungsi ini murni hanya melakukan satu hingga dua operasi penugasan dan perbandingan aritmatika pointer dasar tanpa memerlukan iterasi maupun

alokasi struktur data baru, ketiganya dieksekusi secara instan dengan kompleksitas waktu **O(1)** serta kompleksitas ruang tambahan sebesar **O(1)**.

Sementara itu, fungsi operasional yakni *push*, *pop*, dan *peek* dieksekusi dengan presisi aritmatika pointer yang serupa setelah melewati validasi keamanan. Alur fungsi *push* berjalan dengan menyimpan data tepat di alamat pointer *top* saat ini, lalu memajukan pointer satu langkah; ini terbukti aman karena memastikan elemen tersimpan tanpa menimpa data lain dan membiarkan *top* selalu siap di slot kosong berikutnya. Sebaliknya, fungsi *pop* bekerja sangat efisien dengan cara "menghapus" elemen teratas secara logis, yaitu murni dengan memundurkan pointer *top* satu langkah, sehingga elemen lama akan otomatis tertimpa pada siklus masuk berikutnya tanpa perlu manipulasi memori secara eksplisit. Terakhir, fungsi *peek* membaca data persis satu langkah di belakang pointer *top* ($top - 1$), yang kebenarannya mutlak karena data valid terakhir pasti menetap tepat di bawah slot kosong yang ditunjuk pointer saat ini. Keseluruhan manipulasi nilai dan akses memori langsung ini tidak melibatkan perulangan apapun, sehingga kompleksitas waktu dan ruang untuk ketiga fungsi tersebut masing-masing bernilai mutlak **O(1)**.

SOAL 2

Time Limit: 1.0s Memory Limit: 64 MB

Si Palui menemukan sebuah mesin hitung tua di gudang. Mesin tersebut memiliki cara kerja yang tidak biasa. Alih-alih menggunakan format perhitungan seperti yang biasa kita kenal, mesin ini membaca input berupa deretan simbol yang dipisahkan oleh spasi.

Setiap simbol bisa berupa:

- sebuah bilangan bulat, atau
- sebuah operator (+, -, *, /)

Cara kerja mesin adalah sebagai berikut:

- Mesin membaca simbol dari kiri ke kanan.
- Jika simbol adalah bilangan, maka bilangan tersebut disimpan ke dalam memori mesin.
- Jika simbol adalah operator, maka mesin akan mengambil dua nilai yang paling baru dimasukkan ke dalam memori yang disimpan, melakukan operasi sesuai operator tersebut, lalu menyimpan kembali hasilnya ke dalam memori.
- Setelah seluruh simbol diproses, akan tersisa satu nilai di dalam memori. Nilai inilah yang ditampilkan sebagai hasil akhir.

Sebagai asisten Si Palui, tugas Anda adalah membantu menghitung hasil dari mesin tersebut.

Batasan

$$1 \leq n \leq 100$$

$$-1000 \leq A_i \leq 1000$$

Keterangan:

- n adalah jumlah simbol dalam ekspresi

- A_i adalah nilai bilangan pada input

Simbol yang valid hanya:

bilangan bulat

operator: +, -, *, /

Format Input

n

A1 A2 ... An

Keterangan:

Baris pertama n menyatakan jumlah atau panjang simbol

Baris kedua berisi simbol A sebanyak n buah

Format Output

Sebuah bilangan hasil dari kalkulator Si Palui.

Contoh Kasus

Input	Output
2 2 3 +	5
9 5 1 2 + 4 * + 3 -	14
5 10 2 / 3 *	15

A. Source Code

Tabel 2 Source Code prob1.cpp

```

1  #include <iostream>
2  #include <string>
3  #include "stack.h"
4
5  using namespace std;
```

```

6
7 int main() {
8     int n;
9     if (!(cin >> n)) return 0;
10
11     Stack s;
12     init(&s);
13
14     for (int i = 0; i < n; ++i) {
15         string token;
16         cin >> token;
17
18         if (token == "+" || token == "-" || token
19 == "*" || token == "/") {
20             int b = peek(&s);
21             pop(&s);
22
23             int a = peek(&s);
24             pop(&s);
25
26             int result = 0;
27             if (token == "+") {
28                 result = a + b;
29             } else if (token == "-") {
30                 result = a - b;
31             } else if (token == "*") {
32                 result = a * b;
33             } else if (token == "/") {
34                 result = a / b;
35             }
36         }
37     }
38 }

```

```

35
36         push(&s, result);
37     } else {
38         int nilai = stoi(token);
39         push(&s, nilai);
40     }
41 }
42
43 cout << peek(&s) << endl;
44
45 return 0;
46 }

```

B. Output Program

```

C:\Users\LENOVO\Documents\GitHub\module-2-stack-and-queue-wyattPNG>. \prob1.exe
3
2 3 +
5
9
C:\Users\LENOVO\Documents\GitHub\module-2-stack-and-queue-wyattPNG>. \prob1.exe
5 1 2 + 4 * + 3 -
14
C:\Users\LENOVO\Documents\GitHub\module-2-stack-and-queue-wyattPNG>. \prob1.exe
5
10 2 / 3 *
15
C:\Users\LENOVO\Documents\GitHub\module-2-stack-and-queue-wyattPNG>

```

Gambar 1. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Pada baris [1, 2, dan 3], syntax `#include <iostream>`, `#include <string>`, dan `#include "stack.h"` berfungsi untuk mengimpor library input-output, library

manipulasi teks, dan file header struktur data stack lokal, pada baris [5] syntax *using namespace std;* berfungsi untuk mempersingkat penulisan perintah standar C++, lalu pada baris [7] syntax *int main()* berfungsi untuk mendeklarasikan eksekusi fungsi utama program. Pada baris [8 dan 9], syntax *int n;* dan *cin >> n* berfungsi untuk menerima jumlah panjang input sekaligus menghentikan program jika masukan kosong, pada baris [11 dan 12] syntax *Stack s;* dan *init(&s);* berfungsi untuk mendeklarasikan dan menyiapkan tumpukan memori, lalu pada baris [14] syntax *for* berfungsi untuk mengulang proses pembacaan simbol-simbol kalkulator sebanyak n kali. Kemudian pada baris [15 dan 16], syntax *string token;* dan *cin >> token;* berfungsi untuk menerima dan membaca input dari terminal sebagai teks, pada baris [18] syntax kondisi *if* berfungsi untuk mendeteksi apakah teks tersebut merupakan sebuah operator aritmatika, lalu pada baris [19 hingga 23] penggunaan fungsi *peek(&s)* dan *pop(&s)* berfungsi untuk mengambil serta menghapus dua angka paling atas di dalam stack untuk dijadikan bahan hitungan. Pada baris [26 hingga 34], syntax percabangan *if-else* lanjutan berfungsi untuk melakukan perhitungan matematika nyata pada kedua angka tersebut sesuai jenis operatornya, pada baris [38] syntax *stoi(token)* berfungsi untuk mengubah teks masukan menjadi bilangan integer bulat apabila simbol tersebut bukan sebuah operator, lalu pada baris [36 dan 39] penggunaan fungsi *push(&s, ...)* berfungsi untuk mendorong hasil hitungan atau angka baru tersebut agar masuk kembali ke dalam tumpukan. Terakhir, pada baris [43], syntax *cout << peek(&s)* berfungsi untuk mencetak sisa satu nilai hasil kalkulasi akhir dari dalam stack ke layar terminal, lalu pada baris [45] syntax *return 0;* berfungsi untuk menutup program dan memberi sinyal bahwa eksekusi telah selesai tanpa masalah.

SOAL 3

Pada file queue.h yang diberikan, implementasikan seluruh fungsi yang dideklarasikan ke dalam file queue.cpp. Untuk menangani kondisi tidak valid (invalid case), gunakan mekanisme exception handling dengan throw.

Jelaskan alur algoritma dari setiap fungsi yang diimplementasikan, serta berikan justifikasi atau pembuktian singkat mengenai kebenaran algoritma tersebut. Selain itu, lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi yang ada. (Gunakan Big-O Notation).

A. Source Code

Tabel 3 Source Code queue.cpp

1	#include "queue.h"
2	#include <stdexcept>
3	
4	void init(Queue* q) {
5	q->front = nullptr;
6	q->rear = nullptr;
7	}
8	
9	bool isEmpty(const Queue* q) {
10	return q->front == nullptr;
11	}
12	
13	bool isFull(const Queue* q) {
14	if (isEmpty(q)) return false;
15	
16	const int* next_rear = q->data + ((q->rear -
17	q->data + 1) % MAX);
18	return next_rear == q->front;


```

19 }

20
21 void enqueue(Queue* q, int value) {
22     if (isFull(q)) {
23         throw std::overflow_error("Queue penuh
24         (overflow)");
25     }
26     if (isEmpty(q)) {
27         q->front = q->data;
28         q->rear = q->data;
29     } else {
30         q->rear = q->data + ((q->rear - q->data
31         + 1) % MAX);
32     }
33     *(q->rear) = value;
34 }
35
36 void dequeue(Queue* q) {
37     if (isEmpty(q)) {
38         throw std::underflow_error("Queue kosong
39         (underflow)");
40     }
41     if (q->front == q->rear) {
42         q->front = nullptr;
43         q->rear = nullptr;
44     } else {
45         q->front = q->data + ((q->front - q->data
46         + 1) % MAX);

```

```

46     }

47 }

48
49 int front(const Queue* q) {
50     if (isEmpty(q)) {
51         throw std::underflow_error("Queue kosong
52         (underflow)");
53     }
54     return *(q->front);
55 }
56
57 int back(const Queue* q) {
58     if (isEmpty(q)) {
59         throw std::underflow_error("Queue kosong
60         (underflow)");
61     }
62     return *(q->rear);
63 }

```

B. Pembahasan

Fungsi *init*, *isEmpty*, dan *isFull* bertindak sebagai pengendali status dasar pada struktur antrean ini dengan memanfaatkan penanda *nullptr* dan aritmatika modulo. Alur fungsi *init* menginisialisasi pointer *front* dan *rear* menjadi *nullptr*, yang memberikan justifikasi mutlak bahwa antrean dalam keadaan benar-benar kosong sejak awal tanpa terikat pada sisa indeks memori acak. Kebenaran kondisi ini secara langsung divalidasi oleh fungsi *isEmpty* yang cukup mengevaluasi apakah pointer *front* masih bernilai *nullptr*. Sementara itu, fungsi *isFull* membuktikan ketiadaan sisa ruang dengan menyimulasikan pergeseran pointer *rear* satu langkah ke depan secara melingkar menggunakan rumus modulo persentase kapasitas array; apabila alamat memori pada posisi

masa depan *rear* tersebut sama persis dengan alamat *front* saat ini, maka antrian terbukti penuh dan saling berhimpitan. Karena ketiga fungsi pengendali ini murni hanya melakukan

operasi penugasan pointer, perbandingan, dan perhitungan aritmatika dasar tanpa melibatkan perulangan satu pun, seluruh prosesnya diselesaikan seketika sehingga menghasilkan kompleksitas waktu $O(1)$ dan kompleksitas ruang tambahan sebesar $O(1)$.

Fungsi operasional seperti *enqueue*, *dequeue*, *front*, dan *back* merawat siklus pergerakan memori agar prinsip First-In-First-Out (FIFO) tetap terjaga secara efisien. Pada alur fungsi *enqueue*, algoritma menggeser pointer *rear* ke slot berikutnya secara melingkar (atau menempatkan keduanya di titik awal jika antrian tadinya kosong) lalu menyimpan data tepat pada alamat pointer tersebut; ini dijustifikasi sebagai langkah yang sepenuhnya aman karena elemen baru pasti menempati ruang kosong yang tepat tanpa memicu tumpang tindih berkat validasi *isFull* sebelumnya. Fungsi *dequeue* bekerja dengan konsep logis serupa, di mana elemen terdepan dianggap "terhapus" hanya dengan memajukan pointer *front* menjauhinya satu langkah secara melingkar, atau mengembalikan *front* dan *rear* menjadi *nullptr* apabila itu merupakan data terakhir yang tersisa. Adapun fungsi *front* dan *back* bertugas sebagai jendela pembacaan yang secara pasif mengambil nilai dari alamat memori *front* dan *rear* saat ini melalui operasi dereferensi langsung. Seluruh manipulasi penambahan, penghapusan logis, dan akses memori ini dirancang secara terstruktur tanpa perlu menggeser elemen-elemen array secara fisik di belakang layar, sehingga kompleksitas waktu dan ruang tambahannya terjamin absolut berada pada tingkat $O(1)$ untuk setiap fungsi.

SOAL 4

Time Limit: 1.0s Memory Limit: 64 MB

Si Palui memiliki sebuah deretan angka yang tersusun rapi. Ia tertarik untuk mengamati sebagian angka secara berurutan dengan jumlah tertentu.

Ia memilih sejumlah k angka yang bersebelahan, lalu menghitung jumlahnya. Setelah itu, ia menggeser pilihannya satu langkah ke kanan dan kembali menghitung jumlah dari angka-angka yang terpilih.

Proses ini dilakukan terus hingga tidak memungkinkan lagi untuk memilih k angka berurutan. Sebagai asistennya, bantulah Si Palui untuk menentukan hasil perhitungan tersebut.

Batasan

- $1 \leq n \leq 100$
- $-1 \leq k \leq n$
- $-1000 \leq A_i \leq 1000$

Keterangan:

- n : jumlah elemen
- k : ukuran jendela
- A_i : elemen array

Format Input

n k

A_1 A_2 ... A_n

Keterangan:

- Baris pertama n menyatakan jumlah atau panjang array. Lalu k besar atau jumlah elemen dalam satu kelompok yang dibentuk
- Baris kedua berisi simbol A sebanyak n buah

Format Output

Cetak hasil tiap masing-masing kelompok dalam satu baris dipisahkan dengan spasi

Contoh Kasus

Input	Output
5 3 1 2 3 4 5	6 9 12
5 5 1 2 3 4 5	15
6 2 4 -1 2 -3 5 -1	3 1 -1 2 6

A. Source Code

Tabel 4 Source Code prob2.cpp

```

1  #include <iostream>
2  #include "queue.h"
3
4  using namespace std;
5
6  int main() {
7      int n, k;
8
9      if (!(cin >> n >> k)) return 0;
10
11     if (k <= 0) return 0;
12
13     Queue q;
14     init(&q);
15

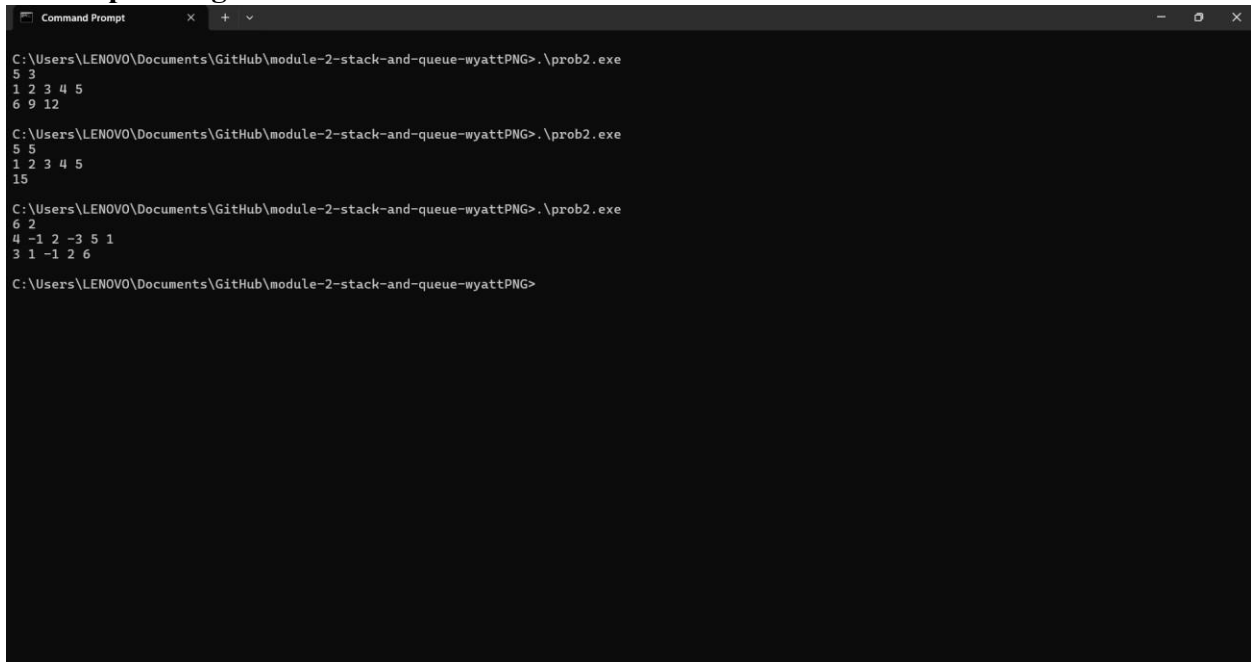
```

```

16     int current_sum = 0;
17
18     for (int i = 0; i < n; ++i) {
19         int val;
20         cin >> val;
21
22         if (i < k) {
23             enqueue(&q, val);
24             current_sum += val;
25
26             if (i == k - 1) {
27                 cout << current_sum;
28             }
29         } else {
30             current_sum -= front(&q);
31             dequeue(&q);
32
33             enqueue(&q, val);
34             current_sum += val;
35
36             cout << " " << current_sum;
37         }
38     }
39
40     cout << endl;
41
42     return 0;
43 }

```

B. Output Program



```
Command Prompt
C:\Users\LENOVO\Documents\GitHub\module-2-stack-and-queue-wyattPNG>.\prob2.exe
5 3
1 2 3 4 5
6 9 12

C:\Users\LENOVO\Documents\GitHub\module-2-stack-and-queue-wyattPNG>.\prob2.exe
5 5
1 2 3 4 5
15

C:\Users\LENOVO\Documents\GitHub\module-2-stack-and-queue-wyattPNG>.\prob2.exe
6 2
4 -1 2 -3 5 1
3 1 -1 2 6

C:\Users\LENOVO\Documents\GitHub\module-2-stack-and-queue-wyattPNG>
```

Gambar 2. Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Pada baris [7 dan 9], syntax `int n, k;` dan `cin >> n >> k` berfungsi untuk mendeklarasikan sekaligus membaca jumlah total elemen array beserta ukuran jendela pengamatan (k). Pada baris [11], syntax `if (k <= 0)` berfungsi sebagai pengaman untuk menghentikan kalkulasi jika ukuran jendela tidak valid. Pada baris [13 dan 14], syntax `Queue q;` dan `init(&q);` berfungsi untuk menyiapkan objek antrean kosong. Memasuki baris [16], syntax `int current_sum = 0;` berfungsi untuk menginisialisasi variabel penyimpan total nilai di dalam jendela saat ini. Pada baris [18 hingga 20], syntax `for` dan `cin >> val;` berfungsi untuk membaca deretan angka secara berurutan sebanyak n kali. Pada baris [22 hingga 28], blok percabangan `if (i < k)` berfungsi untuk membangun fase pertama (pengisian jendela awal) dengan cara memasukkan nilai ke memori antrean menggunakan `enqueue(&q, val)`, menambahkannya ke `current_sum`, dan mencetak total perdana tersebut tepat ketika jendela berukuran k sudah terisi penuh. Apabila jendela sudah melewati batas k, pada baris [29 hingga 37], blok percabangan `else` berfungsi untuk menggeser jendela pengamatan dengan cara mengurangi total jumlahan menggunakan nilai elemen paling lama via `front(&q)`, membuang elemen usang tersebut dari memori antrean

dengan `dequeue(&q)`, memasukkan angka yang baru dibaca ke dalam antrian, serta menambahkan angka baru tersebut ke `current_sum` lalu mencetaknya berdampingan ke layar. Terakhir, pada baris [40], syntax `cout << endl;` berfungsi untuk memberikan jarak baris baru setelah seluruh proses pergeseran jendela selesai.

MODUL 3: LINKED LIST

SOAL 1

General Technical Requirements:

Standard: C++23

Memory Management: Manual (new/delete)

Constraints: Strictly NO STL (<vector>, <list>, dll), NO Global Arrays.

Pada file `single_linked_list.h` yang diberikan, implementasikan struktur `SingleLinkedList` beserta fungsi-fungsinya ke dalam file `single_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 3 Source Code single_linked_list.cpp

1	<code>#include "single_linked_list.h"</code>
2	<code>#include <iostream></code>
3	<code>#include <stdexcept></code>
4	
5	<code>void SingleLinkedList::init() {</code>
6	<code> head = nullptr;</code>
7	<code> tail = nullptr;</code>

```

8         size = 0;
9     }
10
11 bool SingleLinkedList::is_empty() {
12     return size == 0;
13 }
14
15 void SingleLinkedList::add_front(int val) {
16     Node* newNode = new Node{val, nullptr};
17     if (is_empty()) {
18         head = tail = newNode;
19         tail->next = head;
20     } else {
21         newNode->next = head;
22         head = newNode;
23         tail->next = head;
24     }
25     size++;
26 }
27
28 void SingleLinkedList::add_back(int val) {
29     Node* newNode = new Node{val, nullptr};

```

```

30     if (is_empty()) {
31         head = tail = newNode;
32         tail->next = head;
33     } else {
34         tail->next = newNode;
35         tail = newNode;
36         tail->next = head;
37     }
38     size++;
39 }
40
41 void SingleLinkedList::add_idx(int val, int idx)
42 {
43     if (idx < 0 || idx > size) {
44         throw std::out_of_range("Index out of
45         bounds");
46     }
47
48     if (idx == 0) {
49         add_front(val);
50         return;
51     }
52
53     if (idx == size) {

```

```

51         add_back(val);
52         return;
53     }
54
55     Node* newNode = new Node{val, nullptr};
56     Node* curr = head;
57
58     for (int i = 0; i < idx - 1; ++i) {
59         curr = curr->next;
60     }
61
62     newNode->next = curr->next;
63     curr->next = newNode;
64     size++;
65 }
66
67 void SingleLinkedList::delete_front() {
68     if (is_empty()) {
69         throw std::out_of_range("List is
empty");
70     }
71
72     Node* temp = head;

```

```

73     if (size == 1) {
74         head = tail = nullptr;
75     } else {
76         head = head->next;
77         tail->next = head;
78     }
79     delete temp;
80     size--;
81 }
82
83 void SingleLinkedList::delete_back() {
84     if (is_empty()) {
85         throw std::out_of_range("List is
empty");
86     }
87
88     if (size == 1) {
89         delete_front();
90         return;
91     }
92
93     Node* curr = head;
94     while (curr->next != tail) {

```

```

95         curr = curr->next;
96     }
97
98     Node* temp = tail;
99     tail = curr;
100    tail->next = head;
101    delete temp;
102    size--;
103 }
104
105 void SingleLinkedList::delete_idx(int idx) {
106     if (idx < 0 || idx >= size) {
107         throw std::out_of_range("Index out of
108         bounds");
109     }
110
111     if (idx == 0) {
112         delete_front();
113         return;
114     }
115
116     if (idx == size - 1) {
117         delete_back();
118         return;

```



```

117     }
118
119     Node* curr = head;
120     for (int i = 0; i < idx - 1; ++i) {
121         curr = curr->next;
122     }
123
124     Node* temp = curr->next;
125     curr->next = temp->next;
126     delete temp;
127     size--;
128 }
129
130 void SingleLinkedList::display() {
131     if (is_empty()) {
132         std::cout << "List is empty\n";
133         return;
134     }
135
136     Node* curr = head;
137     for (int i = 0; i < size; ++i) {
138         std::cout << curr->data;

```

```

139         if (i < size - 1) std::cout << " ";
140         curr = curr->next;
141     }
142     std::cout << "\n";
143 }
144
145 int SingleLinkedList::get(int idx) {
146     if (idx < 0 || idx >= size) {
147         throw std::out_of_range("Index out of
148         bounds");
149     }
150     Node* curr = head;
151     for (int i = 0; i < idx; ++i) {
152         curr = curr->next;
153     }
154     return curr->data;
155 }
156
157 void SingleLinkedList::set(int val, int idx) {
158     if (idx < 0 || idx >= size) {
159         throw std::out_of_range("Index out of
160         bounds");

```

```

160     }
161
162     Node* curr = head;
163     for (int i = 0; i < idx; ++i) {
164         curr = curr->next;
165     }
166     curr->data = val;
167 }
168
169 void SingleLinkedList::clear() {
170     while (!is_empty()) {
171         delete_front();
172     }
173 }

```

B. Pembahasan

Pada baris [1, 2, dan 3] syntax *#include* mengimpor file header lokal dan library standar C++ dengan kompleksitas waktu dan ruang $O(1)$, pada baris [5 hingga 8] syntax *init()* menyetel nilai pointer memori menjadi *nullptr* untuk inisialisasi awal, pada baris [11 dan 12] syntax *is_empty()* mengevaluasi sisa kapasitas elemen list, pada baris [15 dan 28] syntax *add_front()* dan *add_back()* mengalokasikan ruang memori dinamis secara manual di dalam heap menggunakan perintah *new Node*, pada baris [19 dan 36] syntax *tail->next = head* merajut pointer menyilang agar struktur melingkar secara otomatis dengan kompleksitas $O(1)$, lalu pada baris [41, 58, dan 59] syntax *add_idx()* beserta perulangan *for* menelusuri posisi indeks memori secara linier yang membuat algoritma memakan kompleksitas waktu $O(n)$ dengan ruang $O(1)$.

Pada baris [67] syntax *delete_front()* memperbarui pergeseran rujukan awal pointer node, pada baris [79, 100, dan 126] syntax *delete* melepaskan alokasi memori manual kembali ke sistem operasi, pada baris [83 dan 105] syntax *delete_back()* dan *delete_idx()* melakukan proses pencabutan node iteratif berskala waktu $O(n)$ dan ruang $O(1)$, pada baris [130, 145, dan 157] syntax *display()*, *get()*, dan *set()* mengakses serta memanipulasi rentetan data memori, pada baris [137, 151, dan 163] syntax *for* yang dibatasi kapasitas *size* mencegah sistem terjebak siklus melingkar abadi (infinite loop) dengan proses penelusuran linier bernilai $O(n)$, lalu pada baris [169 hingga 171] syntax *clear()* dan *while (!is_empty())* memanggil perintah *delete_front()* secara berkelanjutan yang menjamin algoritma ini sangat valid dalam menghancurkan total seluruh objek *new* secara presisi $O(n)$ demi menghindari kebocoran memori (memory leak).

SOAL 2

Time Limit: 1.0 s
Memory Limit: 64 MB

Si Palui terjebak di dalam sebuah kuil kuno yang dikelilingi oleh N buah batu relikui yang membentuk formasi lingkaran sempurna. Untuk membuka pintu keluar, ia harus menghancurkan batu-batu tersebut dengan urutan tertentu menggunakan pancaran energi beruntun.

Pancaran energi bermula dari batu pertama, lalu melompat sejauh K langkah ke arah depan, lalu menghancurkan batu target tersebut. Namun, kuil ini memiliki anomali energi dinamis:

- Jika nilai ukiran pada batu yang baru saja dihancurkan adalah bilangan genap, maka lompatan berikutnya akan bertambah jauh ($K=K+1$).
- Jika nilai ukiran pada batu yang dihancurkan adalah bilangan ganjil, maka lompatan berikutnya akan melemah ($K=K-1$).
- Jika pada suatu titik energi lompatan habis atau negatif ($K \leq 0$), sistem akan mengalami reset dan mengembalikan nilai lompatan ke nilai K awal (saat pancaran energi pertama kali diinisialisasi).

Setelah seluruh batu hancur kecuali satu, batu terakhir itulah yang akan menjadi kunci pintu keluar.

Batasan

- $1 \leq N \leq 10^4$
- $1 \leq K \leq 10^9$
- $1 \leq A_i \leq 10^6$ (Nilai ukiran pada batu)
- Program wajib menggunakan implementasi struktur data yang dibuat dari Soal 1.

Format Input

N K

A_1 A_2 ... A_N

Format Output

Sebuah bilangan bulat tunggal yang merupakan nilai ukiran pada batu terakhir yang tersisa.

Contoh Kasus

Input	Output
5 2 1 4 3 6 2	6
4 5 10 11 12 13	12

A. Source Code

Tabel 4 Source Code prob_a.cpp

```
1  #include <iostream>
2  #include "single_linked_list.h"
3
4  using namespace std;
5
6  int main() {
7      ios_base::sync_with_stdio(false);
8      cin.tie(NULL);
9
10     int n;
11     int k;
12
13     if (!(cin >> n >> k)) return 0;
```

```

14
15     SingleLinkedList list;
16     list.init();
17
18     for (int i = 0; i < n; i++) {
19         int val;
20         cin >> val;
21         list.add_back(val);
22     }
23
24     int initial_k = k;
25     int current_k = k;
26
27     Node* curr = list.head;
28     Node* prev = list.tail;
29
30     while (list.size > 1) {
31         int steps = (current_k - 1) % list.size;
32
33         for (int i = 0; i < steps; i++) {
34             prev = curr;
35             curr = curr->next;

```

```

36         }
37
38         int destroyed_val = curr->data;
39
40         prev->next = curr->next;
41         if (curr == list.head) list.head = curr-
>next;
42         if (curr == list.tail) list.tail = prev;
43
44         Node* target = curr;
45         curr = curr->next;
46         delete target;
47         list.size--;
48
49         if (destroyed_val % 2 == 0) {
50             current_k++;
51         } else {
52             current_k--;
53         }
54
55         if (current_k <= 0) {
56             current_k = initial_k;
57         }

```

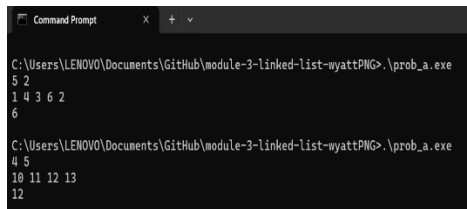


```

58     }
59
60     if (list.size == 1) {
61         cout << list.head->data << "\n";
62     }
63
64     list.clear();
65
66     return 0;
67 }

```

B. Output Program



```

C:\Users\LENOVO\Documents\GitHub\module-3-linked-list-wyatt\prob_a.exe
5 2
1 4 3 6 2
6
C:\Users\LENOVO\Documents\GitHub\module-3-linked-list-wyatt\prob_a.exe
4 5
10 11 12 13
12

```

Gambar 3. Screenshot Hasil Jawaban Soal 2

C. Pembahasan

Pada baris [1 dan 2] syntax `#include <iostream>` dan `#include "single_linked_list.h"` mengimpor library standar C++ serta file header lokal yang memuat struktur data dengan kompleksitas waktu dan ruang $O(1)$, pada baris [4] syntax `using namespace std;` mendeklarasikan penggunaan ruang nama standar, pada baris [6] syntax `int main()` menginisiasi blok fungsi utama eksekusi program, pada baris [7 dan 8] syntax `ios_base::sync_with_stdio(false);` dan `cin.tie(NULL);` mematikan sinkronisasi I/O bawaan demi mempercepat laju pembacaan data, pada baris [10 hingga 13] syntax `cin >> n >> k` digunakan untuk membaca nilai jumlah elemen dan langkah awal sekaligus memvalidasi masukannya, pada baris [15 dan 16] syntax

list.init() memanggil inisialisasi awal senarai melingkar yang kosong, pada baris [18 hingga 22] syntax *list.add_back(val)* di dalam perulangan *for* digunakan untuk memesan dan mengalokasikan ruang memori secara manual ke dalam *heap* untuk tiap elemen baru sehingga memakan kompleksitas waktu $O(N)$ dan ruang $O(N)$, pada baris [24 hingga 28] syntax *Node* curr* dan *Node* prev* memposisikan titik awal penelusuran pointer pada struktur sirkuler, pada baris [30] syntax *while (list.size > 1)* menciptakan perulangan eliminasi utama yang secara keseluruhan membentuk algoritma berjalan kompleksitas waktu $O(N^2)$ karena adanya penelusuran berulang, pada baris [31] syntax *(current_k - 1) % list.size* adalah bentuk optimasi logis krusial yang digunakan untuk memangkas iterasi pencarian berlebih menggunakan operasi modulo yang berjalan secepat $O(1)$, pada baris [33 hingga 36] syntax *for* menggeser pointer penelusuran langkah demi langkah untuk menemukan target eliminasi, pada baris [39 hingga 42] syntax *prev->next = curr->next* melakukan manuver manipulasi pointer $O(1)$ untuk melepaskan target dari rantai senarai tanpa memutus sifat melingkarinya, pada baris [44 hingga 47] syntax *delete target;* digunakan secara tegas untuk menghancurkan node dari area *heap* dan melepaskan memorinya secara manual yang membuktikan validitas algoritma ini dalam menghindari ancaman *memory leak*, pada baris [49 hingga 57] syntax percabangan *if* memodifikasi arah dan sisa energi lompatan *current_k* dengan mengevaluasi kondisi genap atau ganjil secara $O(1)$, pada baris [60 dan 61] syntax *cout << list.head->data* digunakan untuk mencetak satu-satunya node terakhir yang selamat ke layar, pada baris [64] syntax *list.clear()* memanggil fungsi pelucutan memori sapu bersih yang menjamin program ini tuntas dan terbebas sepenuhnya dari kebocoran memori dengan menghancurkan seluruh sisa alokasi objek secara manual, lalu terakhir pada baris [66] syntax *return 0;* digunakan untuk mengakhiri fungsi utama dengan kode keberhasilan ke sistem operasi.

SOAL 3

General Technical Requirements:

Standard: C++23

Memory Management: Manual (new/delete)

Constraints: Strictly NO STL, NO Global Arrays.

Pada file `double_linked_list.h` yang diberikan, implementasikan struktur `DoubleLinkedList` beserta fungsi-fungsinya ke dalam file `double_linked_list.cpp`.

Jelaskan alur algoritma dari setiap metode yang diimplementasikan. Pastikan metode `clear()` berjalan dengan benar untuk menghindari memory leak (kebocoran memori). Lakukan analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Source Code

Tabel 5 Source Code double_linked_list.cpp

1	<code>#include "double_linked_list.h"</code>
2	<code>#include <iostream></code>
3	<code>#include <stdexcept></code>
4	
5	<code>void DoubleLinkedList::init() {</code>
6	<code> head = nullptr;</code>
7	<code> tail = nullptr;</code>

```

8         size = 0;
9     }
10
11 bool DoubleLinkedList::is_empty() {
12     return size == 0;
13 }
14
15 void DoubleLinkedList::add_front(int val) {
16     Node*      newNode      =      new
Node{static_cast<char>(val), nullptr, nullptr};
17
18     if (is_empty()) {
19         head = tail = newNode;
20         head->next = head;
21         head->prev = head;
22     } else {
23         newNode->next = head;
24         newNode->prev = tail;
25         head->prev = newNode;
26         tail->next = newNode;
27         head = newNode;
28     }
29     size++;

```

```

30 }
31
32 void DoubleLinkedList::add_back(int val) {
33     Node*      newNode      =      new
Node{static_cast<char>(val), nullptr, nullptr};
34
35     if (is_empty()) {
36         head = tail = newNode;
37         head->next = head;
38         head->prev = head;
39     } else {
40         newNode->next = head;
41         newNode->prev = tail;
42         tail->next = newNode;
43         head->prev = newNode;
44         tail = newNode;
45     }
46     size++;
47 }
48
49 void DoubleLinkedList::add_idx(int val, int idx)
{
50     if (idx < 0 || idx > size) {

```

```

51         throw std::out_of_range("Index out of
bounds");
52     }
53
54     if (idx == 0) {
55         add_front(val);
56         return;
57     }
58     if (idx == size) {
59         add_back(val);
60         return;
61     }
62
63     Node* curr = head;
64     for (int i = 0; i < idx; ++i) {
65         curr = curr->next;
66     }
67
68     Node* newNode = new
Node{static_cast<char>(val), curr, curr->prev};
69     curr->prev->next = newNode;
70     curr->prev = newNode;
71     size++;

```

```

72     }
73
74     void DoubleLinkedList::delete_front() {
75         if (is_empty()) {
76             throw std::out_of_range("Index out of
77             bounds");
78         }
79         Node* temp = head;
80         if (size == 1) {
81             head = tail = nullptr;
82         } else {
83             head = head->next;
84             head->prev = tail;
85             tail->next = head;
86         }
87         delete temp;
88         size--;
89     }
90
91     void DoubleLinkedList::delete_back() {
92         if (is_empty()) {

```

```

93         throw std::out_of_range("Index out of
bounds");
94     }
95
96     Node* temp = tail;
97     if (size == 1) {
98         head = tail = nullptr;
99     } else {
100         tail = tail->prev;
101         tail->next = head;
102         head->prev = tail;
103     }
104     delete temp;
105     size--;
106 }
107
108 void DoubleLinkedList::delete_idx(int idx) {
109     if (idx < 0 || idx >= size) {
110         throw std::out_of_range("Index out of
bounds");
111     }
112
113     if (idx == 0) {

```



```

114         delete_front();
115         return;
116     }
117     if (idx == size - 1) {
118         delete_back();
119         return;
120     }
121
122     Node* curr = head;
123     for (int i = 0; i < idx; ++i) {
124         curr = curr->next;
125     }
126
127     curr->prev->next = curr->next;
128     curr->next->prev = curr->prev;
129     delete curr;
130     size--;
131 }
132
133 void DoubleLinkedList::display() {
134     if (is_empty()) {
135         std::cout << "List is empty\n";

```

```

136         return;
137     }
138
139     Node* curr = head;
140     for (int i = 0; i < size; ++i) {
141         std::cout << curr->data;
142         if (i < size - 1) std::cout << " ";
143         curr = curr->next;
144     }
145     std::cout << "\n";
146 }
147
148 char DoubleLinkedList::get(int idx) {
149     if (idx < 0 || idx >= size) {
150         throw std::out_of_range("Index out of
151         bounds");
152     }
153
154     Node* curr = head;
155     for (int i = 0; i < idx; ++i) {
156         curr = curr->next;
157     }
158     return curr->data;

```

```

158 }
159
160 void DoubleLinkedList::set(char val, int idx) {
161     if (idx < 0 || idx >= size) {
162         throw std::out_of_range("Index out of
163         bounds");
164     }
165     Node* curr = head;
166     for (int i = 0; i < idx; ++i) {
167         curr = curr->next;
168     }
169     curr->data = val;
170 }
171
172 void DoubleLinkedList::clear() {
173     while (!is_empty()) {
174         delete_front();
175     }
176 }

```

B. Pembahasan

Pada baris [1 hingga 3] syntax *#include* mengimpor file header lokal dan library standar C++ untuk penanganan eksepsi dan input-output dengan kompleksitas waktu dan

ruang $O(1)$, pada baris [5 hingga 9] syntax *init()* menyetel nilai pointer memori menjadi *nullptr* serta *size* menjadi 0 untuk inisialisasi awal, pada baris [11 dan 12] syntax *is_empty()* mengevaluasi sisa kapasitas elemen list, pada baris [15 dan 32] syntax *add_front()* dan *add_back()* mengalokasikan ruang memori dinamis secara manual di dalam heap menggunakan perintah *new Node*, pada baris [20, 21, 37, dan 38] syntax *head->next = head* dan *head->prev = head* beserta modifikasi pointer di dalam blok *else* merajut pointer menyilang secara dua arah agar struktur melingkar ganda beroperasi otomatis dengan kompleksitas $O(1)$, lalu pada baris [49, 64, dan 65] syntax *add_idx()* beserta perulangan *for* menelusuri posisi indeks memori secara linier yang membuat algoritma penyisipan ini memakan kompleksitas waktu $O(n)$ dengan penggunaan ruang $O(1)$.

Pada baris [74 dan 91] syntax *delete_front()* dan *delete_back()* memperbarui pergeseran rujukan awal dan akhir pointer node secara instan senilai $O(1)$, pada baris [87, 104, dan 129] syntax *delete* bertugas esensial melepaskan alokasi memori manual kembali ke sistem operasi dari area heap, pada baris [108, 124, dan 125] syntax *delete_idx()* menelusuri rantai memori menggunakan perulangan untuk melakukan pemutusan pointer node berskala waktu $O(n)$ dan ruang $O(1)$, pada baris [133, 148, dan 160] syntax *display()*, *get()*, dan *set()* mengakses serta memanipulasi rentetan karakter data memori, pada baris [140, 154, dan 166] syntax *for* yang secara ketat dibatasi oleh nilai properti *size* digunakan untuk mencegah sistem terjebak di dalam siklus melingkar abadi (infinite loop) melalui proses penelusuran linier bernilai $O(n)$, lalu pada baris [172 hingga 175] syntax *clear()* dan *while (!is_empty())* memanggil perintah *delete_front()* secara berkelanjutan yang mengonfirmasi bahwa algoritma ini sangat valid dan diyakini benar dalam menghancurkan total seluruh objek bentukan *new* secara terstruktur dengan kompleksitas $O(n)$ demi menghindari bencana kebocoran memori (memory leak).

SOAL 4

Si Palui mengamati fenomena aneh di atas sebuah gelanggang melingkar yang berisi N karakter. Terdapat dua proyektor anomali, Alpha (dimulai dari head, bergerak maju menggunakan next) dan Beta (dimulai dari tail, bergerak mundur menggunakan prev).

Palui melakukan observasi selama R ronde. Pada setiap ronde:

- Proyektor Alpha dipaksa melompat maju sebanyak X langkah. Proyektor Beta dipaksa melompat mundur sebanyak Y langkah (simultan).
- Tabrakan Singular: Jika Alpha dan Beta mendarat di titik atau karakter yang sama persis secara bersamaan, karakter tersebut akan hancur dan lenyap dari lingkaran dimensi. Jika ini terjadi, Alpha berpindah ke karakter next dari karakter yang hancur, dan Beta berpindah ke karakter prev dari karakter yang hancur.
- Resonansi Bersebelahan (Adjacent): Jika Alpha dan Beta saling bersebelahan langsung, sifat kuantum dari kedua karakter tersebut saling bertukar (data antara kedua node tersebut di-swap).

Tentukan sisa karakter pada lingkaran dimensi setelah R ronde selesai dieksekusi.

Batasan

- $1 \leq N \leq 5000$
- $1 \leq R \leq 5000$
- $1 \leq X, Y \leq 1018$
- Struktur wajib menggunakan implementasi struktur data dari Soal 3.

Format Input

N R

C1 C2 ... CN

X1 Y1

X2 Y2

...

XR YR

Keterangan: *Ci* adalah satu karakter char tunggal.

Format Output

Karakter yang tersisa, dicetak berurutan mulai dari head hingga ke elemen terakhir, tanpa spasi. Jika dimensi menjadi kosong, cetak EMPTY.

Contoh Kasus

Input	Output
4 2 A B C D 1 1 2 3	ACB
5 3 V W X Y Z 100000 100000 2 2 5 5	VWYZ

A. Source Code

Tabel 6 Source Code prob_b.cpp

1	#include <iostream>
2	#include <utility>
3	#include "double_linked_list.h"
4	

```
5 using namespace std;
6
7 int main() {
8     ios_base::sync_with_stdio(false);
9     cin.tie(NULL);
10
11     int n, r;
12     if (!(cin >> n >> r)) return 0;
13
14     DoubleLinkedList list;
15     list.init();
16
17     for (int i = 0; i < n; i++) {
18         char c;
19         cin >> c;
20         list.add_back(c);
21     }
22
23     if (list.size == 0) {
24         cout << "EMPTY\n";
25         return 0;
26     }
```

```

27
28     Node* alpha = list.head;
29     Node* beta = list.tail;
30
31     for (int i = 0; i < r; i++) {
32         long long x, y;
33         cin >> x >> y;
34
35         if (list.size == 0) continue;
36
37         long long moves_alpha = x % list.size;
38         if (moves_alpha > list.size / 2) {
39             long long back_moves = list.size -
moves_alpha;
40             for (long long j = 0; j < back_moves;
j++) alpha = alpha->prev;
41         } else {
42             for (long long j = 0; j < moves_alpha;
j++) alpha = alpha->next;
43         }
44
45         long long moves_beta = y % list.size;
46         if (moves_beta > list.size / 2) {

```



```

47         long long forward_moves = list.size
- moves_beta;

48         for (long long j = 0; j <
forward_moves; j++) beta = beta->next;

49     } else {

50         for (long long j = 0; j < moves_beta;
j++) beta = beta->prev;

51     }

52

53     if (alpha == beta) {

54         Node* target = alpha;

55

56         if (list.size == 1) {

57             list.head = nullptr;

58             list.tail = nullptr;

59             alpha = nullptr;

60             beta = nullptr;

61         } else {

62             Node* next_a = target->next;

63             Node* next_b = target->prev;

64

65             target->prev->next = target-
>next;

```

```

66         target->next->prev = target-
>prev;
67
68         if (target == list.head) list.head
= target->next;
69         if (target == list.tail) list.tail
= target->prev;
70
71         alpha = next_a;
72         beta = next_b;
73     }
74
75     delete target;
76     list.size--;
77
78     } else {
79         bool is_adjacent = false;
80
81         if (alpha->next == beta && alpha !=
list.tail) is_adjacent = true;
82         if (beta->next == alpha && beta !=
list.tail) is_adjacent = true;
83
84         if (is_adjacent) {

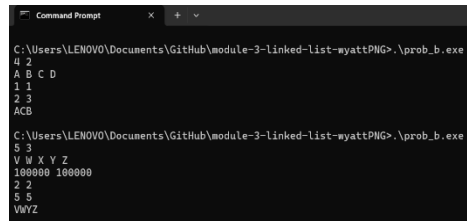
```

```

85         swap(alpha->data, beta->data);
86     }
87 }
88 }
89
90 if (list.size == 0) {
91     cout << "EMPTY\n";
92 } else {
93     Node* curr = list.head;
94     for (int i = 0; i < list.size; i++) {
95         cout << curr->data;
96         curr = curr->next;
97     }
98     cout << "\n";
99 }
100
101 list.clear();
102
103 return 0;
104 }

```

B. Output Program



```
Command Prompt
C:\Users\LENOVO\Documents\GitHub\module-3-linked-list-wyatt\PMG>.\prob.b.exe
4 2
A B C D
1 1
2 3
ACB

C:\Users\LENOVO\Documents\GitHub\module-3-linked-list-wyatt\PMG>.\prob.b.exe
5 3
V W X Y Z
100000 100000
2 2
5 5
VWYZ
```

Gambar 4. Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Pada baris [1 hingga 3] syntax `#include` mengimpor library standar C++ dan file header lokal pembentuk struktur senarai dengan kompleksitas waktu dan ruang $O(1)$, pada baris [7 hingga 12] syntax `int main()` menginisiasi program serta membaca variabel masukan simulasi dengan I/O yang dioptimasi, pada baris [14 hingga 21] syntax `list.add_back(c)` digunakan di dalam perulangan untuk mengalokasikan ruang memori dinamis secara manual ke dalam area heap bagi setiap elemen masukan guna membangun senarai berantai dengan kompleksitas waktu dan ruang $O(N)$, pada baris [28 dan 29] syntax `Node* alpha` dan `Node* beta` menempatkan posisi awal penelusuran proyektor, pada baris [31] syntax `for` menciptakan siklus utama simulasi ronde berjalan kompleksitas waktu $O(R)$, pada baris [37 dan 45] syntax `% list.size` adalah optimasi matematis krusial untuk memangkas pergerakan melingkar berlebih menggunakan operasi modulo yang dieksekusi konstan secepat $O(1)$, pada baris [38 hingga 51] syntax penelusuran pointer `prev` atau `next` menentukan rute langkah terpendek pergerakan secara proporsional bernilai waktu $O(N)$ dan ruang $O(1)$, pada baris [53] syntax `if (alpha == beta)` mengevaluasi kondisi tabrakan singular secara instan $O(1)$, pada baris [62 hingga 72] syntax modifikasi pointer seperti `target->prev->next = target->next` bermanuver memisahkan target dari rantai secara $O(1)$ tanpa merusak integrasi strukturnya, pada baris [75] syntax `delete target;` bertugas esensial menghancurkan node tersebut secara manual dari heap untuk membebaskan ruang memori, pada baris [78 hingga 87] syntax blok `else` mendeteksi resonansi bersebelahan dengan syarat spesifik `alpha != list.tail` guna menukar sifat data menggunakan fungsi `swap` berkecepatan $O(1)$ tanpa menyalahi batasan ujung dimensi linier, pada baris [90 hingga 99] syntax `cout` dan perulangan `for` mencetak karakter yang tersisa beruntun dari `head` berbekal kompleksitas $O(N)$, lalu pada

baris [101] syntax `list.clear()` memusnahkan keseluruhan sisa memori node secara sapu bersih yang menjamin kemutlakan validitas algoritma ini dalam menghindari ancaman kebocoran memori (memory leak) sebelum program diakhiri dengan syntax `return 0;` pada baris [103].

MODUL 4: SORTING

KOMPLEKSITAS WAKTU

A. Source Code

Tabel 7 Source Code sorting_algorithms.cpp

```
174 #include "sorting_algorithms.h"
175 #include <algorithm>
176 #include <chrono>
177
178 using Clock =
    std::chrono::high_resolution_clock;
179
180 void bubble_sort(std::vector<int>& data,
    Metrics& m) {
181     int n = data.size();
182     auto start = Clock::now();
183
184     for (int i = 0; i < n - 1; ++i) {
185         bool swapped = false;
186         for (int j = 0; j < n - i - 1; ++j) {
187             m.comparisons++;
188             if (data[j] > data[j + 1]) {
189                 std::swap(data[j], data[j + 1]);
```

```

190         m.swaps++;
191         swapped = true;
192     }
193 }
194     if (!swapped) break;
195 }
196
197     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
198 }
199
200 void    selection_sort(std::vector<int>&    data,
Metrics& m) {
201     int n = data.size();
202     auto start = Clock::now();
203
204     for (int i = 0; i < n - 1; ++i) {
205         int min_idx = i;
206         for (int j = i + 1; j < n; ++j) {
207             m.comparisons++;
208             if (data[j] < data[min_idx]) {
209                 min_idx = j;
210             }

```

```

211         }
212         if (min_idx != i) {
213             std::swap(data[i], data[min_idx]);
214             m.swaps++;
215         }
216     }
217
218     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
219 }
220
221 void insertion_sort(std::vector<int>& data,
Metrics& m) {
222     int n = data.size();
223     auto start = Clock::now();
224
225     for (int i = 1; i < n; ++i) {
226         int key = data[i];
227         int j = i - 1;
228
229         while (j >= 0) {
230             m.comparisons++;
231             if (data[j] > key) {

```



```

232         data[j + 1] = data[j];
233         m.shifts++;
234         --j;
235     } else {
236         break;
237     }
238 }
239
240     if (j + 1 != i) {
241         data[j + 1] = key;
242         m.shifts++;
243     }
244 }
245
246     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
247 }
248
249 void merge(std::vector<int>& data, int left, int
mid, int right, std::vector<int>& temp, Metrics&
m) {
250     int i = left;
251     int j = mid + 1;

```

```

252     int k = left;
253
254     while (i <= mid && j <= right) {
255         m.comparisons++;
256         if (data[i] <= data[j]) {
257             temp[k++] = data[i++];
258         } else {
259             temp[k++] = data[j++];
260         }
261     }
262
263     while (i <= mid) {
264         temp[k++] = data[i++];
265     }
266     while (j <= right) {
267         temp[k++] = data[j++];
268     }
269
270     for (int p = left; p <= right; ++p) {
271         data[p] = temp[p];
272     }
273 }

```

```

274
275 void merge_sort_helper(std::vector<int>& data,
    int left, int right, std::vector<int>& temp,
    Metrics& m) {
276     m.recursive_calls++;
277     if (left >= right) return;
278
279     int mid = left + (right - left) / 2;
280     merge_sort_helper(data, left, mid, temp, m);
281     merge_sort_helper(data, mid + 1, right,
temp, m);
282     merge(data, left, mid, right, temp, m);
283 }
284
285 void merge_sort(std::vector<int>& data,
    Metrics& m) {
286     auto start = Clock::now();
287
288     if (!data.empty()) {
289         std::vector<int> temp(data.size());
290         merge_sort_helper(data, 0, data.size()
- 1, temp, m);
291     }
292

```

```

293     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
294 }
295
296 int partition(std::vector<int>& data, int low,
int high, Metrics& m) {
297     int pivot = data[high];
298     int i = low - 1;
299
300     for (int j = low; j < high; ++j) {
301         m.comparisons++;
302         if (data[j] < pivot) {
303             ++i;
304             std::swap(data[i], data[j]);
305             m.swaps++;
306         }
307     }
308     std::swap(data[i + 1], data[high]);
309     m.swaps++;
310     return i + 1;
311 }
312

```

```

313 void quick_sort_helper(std::vector<int>& data,
    int low, int high, Metrics& m) {
314     m.recursive_calls++;
315     if (low < high) {
316         int pi = partition(data, low, high, m);
317         quick_sort_helper(data, low, pi - 1, m);
318         quick_sort_helper(data, pi + 1, high,
m);
319     }
320 }
321
322 void quick_sort(std::vector<int>& data,
    Metrics& m) {
323     auto start = Clock::now();
324
325     if (!data.empty()) {
326         quick_sort_helper(data, 0, data.size()
- 1, m);
327     }
328
329     m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
330 }
331

```

```

332 void      radix_sort(std::vector<int>&      data,
Metrics& m) {
333     if (data.empty()) return;
334     auto start = Clock::now();
335
336     int n = data.size();
337
338     int max_val = data[0];
339     for (int i = 1; i < n; ++i) {
340         if (data[i] > max_val) {
341             max_val = data[i];
342         }
343     }
344
345     std::vector<int> output(n);
346
347     for (int exp = 1; max_val / exp > 0; exp *=
10) {
348         int count[10] = {0};
349
350         for (int i = 0; i < n; ++i) {
351             int digit = (data[i] / exp) % 10;
352             count[digit]++;

```

```

353         m.array_accesses++;
354     }
355
356     for (int i = 1; i < 10; ++i) {
357         count[i] += count[i - 1];
358         m.array_accesses += 2;
359     }
360
361     for (int i = n - 1; i >= 0; --i) {
362         int digit = (data[i] / exp) % 10;
363         output[count[digit] - 1] = data[i];
364         m.array_accesses += 2;
365
366         count[digit]--;
367         m.array_accesses++;
368     }
369
370     for (int i = 0; i < n; ++i) {
371         data[i] = output[i];
372         m.array_accesses++;
373     }
374 }

```

```

375
376         m.time_ms = std::chrono::duration<double,
std::milli>(Clock::now() - start).count();
377     }

```

B. Output

```
>> Bubble Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Bubble Sort	Random	100	4935	2564	0	0	0	0.079360
Bubble Sort	Sorted	100	99	0	0	0	0	0.000826
Bubble Sort	Reverse Sorted	100	4950	4950	0	0	0	0.090805
Bubble Sort	Nearly Sorted	100	4719	296	0	0	0	0.033320
Bubble Sort	Many Duplicates	100	4845	2315	0	0	0	0.065093
Bubble Sort	Random	1000	497154	243190	0	0	0	7.332131
Bubble Sort	Sorted	1000	999	0	0	0	0	0.007847
Bubble Sort	Reverse Sorted	1000	499500	499500	0	0	0	10.381107
Bubble Sort	Nearly Sorted	1000	481922	25778	0	0	0	3.368129
Bubble Sort	Many Duplicates	1000	496340	240808	0	0	0	6.934941
Bubble Sort	Random	10000	49993404	25118148	0	0	0	775.899610
Bubble Sort	Sorted	10000	9999	0	0	0	0	0.062448
Bubble Sort	Reverse Sorted	10000	49995000	49995000	0	0	0	851.240321
Bubble Sort	Nearly Sorted	10000	49937030	3119352	0	0	0	364.871891
Bubble Sort	Many Duplicates	10000	49993404	25093086	0	0	0	777.910556

Gambar 5. Screenshot Hasil Bubble Sort

```
>> Selection Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Selection Sort	Random	100	4950	95	0	0	0	0.031458
Selection Sort	Sorted	100	4950	0	0	0	0	0.025763
Selection Sort	Reverse Sorted	100	4950	50	0	0	0	0.037116
Selection Sort	Nearly Sorted	100	4950	4	0	0	0	0.036827
Selection Sort	Many Duplicates	100	4950	89	0	0	0	0.029571
Selection Sort	Random	1000	499500	995	0	0	0	2.777523
Selection Sort	Sorted	1000	499500	0	0	0	0	2.897355
Selection Sort	Reverse Sorted	1000	499500	500	0	0	0	2.836668
Selection Sort	Nearly Sorted	1000	499500	50	0	0	0	2.852273
Selection Sort	Many Duplicates	1000	499500	983	0	0	0	2.862261
Selection Sort	Random	10000	49995000	9992	0	0	0	284.097713
Selection Sort	Sorted	10000	49995000	0	0	0	0	259.839898
Selection Sort	Reverse Sorted	10000	49995000	5000	0	0	0	257.231242
Selection Sort	Nearly Sorted	10000	49995000	500	0	0	0	249.263102
Selection Sort	Many Duplicates	10000	49995000	9981	0	0	0	258.719448

Gambar 6. Screenshot Hasil Selection Sort

```
>> Insertion Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Insertion Sort	Random	100	2656	0	2659	0	0	0.022358
Insertion Sort	Sorted	100	99	0	0	0	0	0.000743
Insertion Sort	Reverse Sorted	100	4950	0	5049	0	0	0.044183
Insertion Sort	Nearly Sorted	100	395	0	375	0	0	0.003583
Insertion Sort	Many Duplicates	100	2412	0	2403	0	0	0.025474
Insertion Sort	Random	1000	244181	0	244184	0	0	2.018444
Insertion Sort	Sorted	1000	999	0	0	0	0	0.006445
Insertion Sort	Reverse Sorted	1000	499500	0	500499	0	0	4.099341
Insertion Sort	Nearly Sorted	1000	26775	0	26766	0	0	0.246953
Insertion Sort	Many Duplicates	1000	241801	0	241794	0	0	2.030349
Insertion Sort	Random	10000	25128135	0	25128140	0	0	209.240317
Insertion Sort	Sorted	10000	9999	0	0	0	0	0.105795
Insertion Sort	Reverse Sorted	10000	49995000	0	50004999	0	0	394.515770
Insertion Sort	Nearly Sorted	10000	3129351	0	3129317	0	0	24.829889
Insertion Sort	Many Duplicates	10000	25103078	0	25103073	0	0	214.161091

Gambar 7. Screenshot Hasil Insertion Sort

```
>> Merge Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time(ms)
Merge Sort	Random	100	535	0	0	199	0	0.018419
Merge Sort	Sorted	100	356	0	0	199	0	0.011511
Merge Sort	Reverse Sorted	100	316	0	0	199	0	0.014562
Merge Sort	Nearly Sorted	100	453	0	0	199	0	0.017384
Merge Sort	Many Duplicates	100	536	0	0	199	0	0.015512
Merge Sort	Random	1000	8690	0	0	1999	0	0.220705
Merge Sort	Sorted	1000	5044	0	0	1999	0	0.135983
Merge Sort	Reverse Sorted	1000	4932	0	0	1999	0	0.134400
Merge Sort	Nearly Sorted	1000	7628	0	0	1999	0	0.170899
Merge Sort	Many Duplicates	1000	8679	0	0	1999	0	0.256779
Merge Sort	Random	10000	120465	0	0	19999	0	3.045771
Merge Sort	Sorted	10000	69008	0	0	19999	0	1.810631
Merge Sort	Reverse Sorted	10000	64608	0	0	19999	0	1.833760
Merge Sort	Nearly Sorted	10000	111524	0	0	19999	0	2.134210
Merge Sort	Many Duplicates	10000	120461	0	0	19999	0	2.921697

Gambar 8. Screenshot Hasil Merge Sort


```
>> Quick Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time (ms)
Quick Sort	Random	100	678	394	0	129	0	0.012383
Quick Sort	Sorted	100	4950	5049	0	199	0	0.078385
Quick Sort	Reverse Sorted	100	4950	2549	0	199	0	0.049081
Quick Sort	Nearly Sorted	100	2638	2607	0	191	0	0.042113
Quick Sort	Many Duplicates	100	909	236	0	181	0	0.010316
Quick Sort	Random	1000	10537	6125	0	1341	0	0.191758
Quick Sort	Sorted	1000	499500	500499	0	1999	0	7.867884
Quick Sort	Reverse Sorted	1000	499500	250499	0	1999	0	4.774263
Quick Sort	Nearly Sorted	1000	35189	25081	0	1611	0	0.469555
Quick Sort	Many Duplicates	1000	12685	4920	0	1801	0	0.160603
Quick Sort	Random	10000	147606	77996	0	13315	0	2.355100
Quick Sort	Sorted	10000	49995000	50004999	0	19999	0	778.423662
Quick Sort	Reverse Sorted	10000	49995000	25004999	0	19999	0	476.603150
Quick Sort	Nearly Sorted	10000	622473	458547	0	16511	0	7.897834
Quick Sort	Many Duplicates	10000	181016	67083	0	18003	0	2.164176

Gambar 9. Screenshot Hasil Quick Sort

```
>> Radix Sort
```

Algorithm	Input Type	N	Comparisons	Swaps	Shifts	Rec.Calls	Arr.Access	Time (ms)
Radix Sort	Random	100	0	0	0	0	1554	0.009701
Radix Sort	Sorted	100	0	0	0	0	1554	0.009032
Radix Sort	Reverse Sorted	100	0	0	0	0	1554	0.014145
Radix Sort	Nearly Sorted	100	0	0	0	0	1554	0.014270
Radix Sort	Many Duplicates	100	0	0	0	0	1036	0.006910
Radix Sort	Random	1000	0	0	0	0	20072	0.114626
Radix Sort	Sorted	1000	0	0	0	0	20072	0.126449
Radix Sort	Reverse Sorted	1000	0	0	0	0	20072	0.108506
Radix Sort	Nearly Sorted	1000	0	0	0	0	20072	0.108553
Radix Sort	Many Duplicates	1000	0	0	0	0	15054	0.083055
Radix Sort	Random	10000	0	0	0	0	250090	1.383037
Radix Sort	Sorted	10000	0	0	0	0	250090	1.414892
Radix Sort	Reverse Sorted	10000	0	0	0	0	250090	1.528683
Radix Sort	Nearly Sorted	10000	0	0	0	0	250090	1.472556
Radix Sort	Many Duplicates	10000	0	0	0	0	200072	1.143662

Gambar 10. Screenshot Hasil Radix Sort

C. Bubble Sort

Best case Bubble Sort mencapai $O(n)$ ketika data sudah terurut. Dengan optimasi flag, algoritma hanya perlu melakukan satu pass tanpa menemukan satu pun pertukaran, sehingga langsung berhenti. Hal ini dikonfirmasi oleh eksperimen: pada $N=10.000$ dengan input Sorted, hanya diperlukan 9.999 comparisons dan 0 swap dengan waktu 0,062 ms.

Average case berada pada $O(n^2)$. Data acak menyebabkan setiap elemen rata-rata harus dibandingkan dan ditukar sebanyak $n/2$ kali, menghasilkan sekitar $n^2/4$ operasi. Pada $N=10.000$ dengan input Random, tercatat 49.993.404 comparisons dan 25.118.148 swaps dengan waktu 775,90 ms.

Worst case $O(n^2)$ terjadi pada data terurut terbalik. Setiap iterasi membutuhkan perbandingan dan pertukaran penuh sebanyak $n(n-1)/2$ operasi. Eksperimen menunjukkan 49.995.000 comparisons dan 49.995.000 swaps (sama besar) dengan waktu 851,24 ms untuk $N=10.000$ Reverse Sorted.

D. Selection Sort

Best, average, dan worst case Selection Sort semuanya $O(n^2)$. Selection Sort melakukan jumlah comparisons yang sama, yaitu $n(n-1)/2$, karena harus mencari minimum di seluruh sisa array pada setiap iterasi. Eksperimen mengkonfirmasi hal ini: comparisons selalu

49.995.000 untuk semua jenis input pada $N=10.000$. Yang bervariasi hanya jumlah swaps: input Sorted menghasilkan 0 swap, sementara input Random menghasilkan 9.992 swap.

E. Insertion Sort

Best case $O(n)$ terjadi ketika data sudah terurut. Setiap elemen hanya dibandingkan sekali dengan elemen sebelumnya, menghasilkan $n-1$ comparisons dan 0 shifts. Eksperimen menunjukkan 9.999 comparisons dan waktu 0,11 ms untuk $N=10.000$ Sorted.

Average case $O(n^2)$ pada data acak, karena rata-rata setiap elemen harus digeser melewati $n/4$ elemen. Tercatat 25.128.135 comparisons dan 25.128.140 shifts (waktu 209,24 ms) untuk $N=10.000$ Random.

Worst case $O(n^2)$ pada data terurut terbalik, di mana setiap elemen harus digeser melewati seluruh elemen yang sudah terurut sebelumnya. Eksperimen mencatat 49.995.000 comparisons dan 50.004.999 shifts (waktu 394,52 ms) untuk $N=10.000$ Reverse Sorted. Jumlah shifts sedikit lebih banyak dari comparisons karena ada satu tambahan operasi geser per elemen untuk memindahkan kunci sementara.

F. Merge Sort

Merge Sort memiliki kompleksitas $O(n \log n)$ untuk semua kasus tanpa terkecuali, menjadikannya algoritma yang paling konsisten dan andal. Bahkan pada data sudah terurut, Merge Sort tetap membagi array secara rekursif dan melakukan proses merge, meski dengan jumlah comparisons yang lebih sedikit. Eksperimen menunjukkan comparisons berkisar antara 64.608 (Reverse Sorted) hingga 120.465 (Random) untuk $N=10.000$, semua jauh di bawah algoritma $O(n^2)$. Waktu eksekusi sangat konsisten di rentang 1,81–3,05 ms untuk semua jenis input pada $N=10.000$, membuktikan stabilitasnya.

G. Quick Sort

Best case dan average case Quick Sort adalah $O(n \log n)$, terjadi ketika pivot yang dipilih membelah array secara cukup seimbang. Pada input Random $N=10.000$, hanya diperlukan 147.606 comparisons dengan waktu 2,36 ms.

Worst case $O(n^2)$ terjadi ketika pivot selalu menjadi elemen terkecil atau terbesar, menghasilkan partisi yang sama sekali tidak seimbang (satu sisi kosong). Eksperimen membuktikan hal ini secara nyata: input Sorted dan Reverse Sorted pada $N=10.000$ menghasilkan 49.995.000 comparisons masing-masing dengan waktu 778,42 ms dan 476,60

ms. Recursive calls = $19.999 = 2N-1$, membuktikan degenerasi ke kedalaman stack linear, bukan logaritmik.

H. Radix Sort

Radix Sort memiliki kompleksitas $O(nk)$ untuk semua kasus, di mana k adalah jumlah digit dari nilai maksimum dalam array. Kompleksitasnya tidak bergantung pada distribusi data melainkan hanya pada rentang nilai. Eksperimen mengkonfirmasi hal ini: array accesses selalu 250.090 untuk input Random, Sorted, Reverse Sorted, dan Nearly Sorted pada $N=10.000$, nilai yang identik karena semua jenis input tersebut memiliki rentang nilai yang sama. Input Many Duplicates sedikit lebih rendah (200.072) karena nilai-nilai duplikat yang kecil memerlukan lebih sedikit pass digit.

KARAKTERISTIK ALGORITMA

A. Bubble Sort

Bubble Sort bersifat stable karena pertukaran hanya dilakukan ketika $arr[j] > arr[j+1]$ secara ketat (strictly greater than), sehingga elemen-elemen dengan nilai sama tidak pernah ditukar dan urutan relatifnya tetap terjaga. Algoritma ini juga in-place karena hanya membutuhkan $O(1)$ memori tambahan berupa satu variabel sementara untuk proses pertukaran. Mekanisme kerjanya adalah membandingkan pasangan elemen yang berdekatan secara berulang dari kiri ke kanan, lalu menukarnya jika tidak terurut. Proses ini menyebabkan elemen terbesar akan 'menggelembung' (bubble up) ke posisi akhir array pada setiap pass, seperti gelembung yang naik ke permukaan air.

B. Selection Sort

Selection Sort tidak bersifat stable. Saat menemukan elemen minimum dan menukarnya ke posisi depan, operasi ini dapat mengganggu urutan relatif elemen-elemen dengan nilai yang sama. Sebagai contoh, pada array $[3a, 3b, 1]$, swap antara 1 dan 3a menghasilkan $[1, 3b, 3a]$ yang mengubah urutan relatif antara 3a dan 3b. Algoritma ini bersifat in-place karena hanya membutuhkan $O(1)$ memori tambahan. Mekanisme utamanya adalah pada setiap iterasi ke- i , mencari elemen minimum dari subarray $[i..n-1]$ lalu menukarnya ke posisi i . Keunggulan Selection Sort adalah jumlah swap yang sangat minimal, paling banyak hanya $n-1$ kali, sehingga cocok untuk situasi di mana operasi write ke memori sangat mahal.

C. Insertion Sort

Insertion Sort bersifat stable karena elemen hanya digeser jika secara ketat lebih besar dari kunci yang sedang ditempatkan, sehingga elemen dengan nilai sama mempertahankan urutan relatifnya. Algoritma ini in-place dengan $O(1)$ memori tambahan. Mekanismenya adalah membangun array terurut dari kiri ke kanan: setiap elemen baru 'disisipkan' ke posisi yang tepat di antara elemen-elemen yang sudah terurut dengan cara menggeser elemen-elemen yang lebih besar ke kanan. Ini berbeda dari Bubble Sort yang menukar pasangan adjacent; Insertion Sort menggeser secara efisien menggunakan satu operasi shift (bukan tiga assignment layaknya swap), sehingga konstanta tersembunyinya lebih kecil.

D. Merge Sort

Merge Sort bersifat stable karena saat proses merge, ketika dua elemen memiliki nilai sama, elemen dari subarray kiri selalu dipilih terlebih dahulu sehingga urutan relatif terjaga. Algoritma ini tidak in-place karena membutuhkan $O(n)$ memori tambahan untuk array sementara selama proses penggabungan. Mekanismenya menggunakan strategi Divide and Conquer: array dibagi dua secara rekursif hingga ukuran menjadi 1 (basis rekursi), kemudian digabungkan kembali secara terurut. Proses merge membandingkan elemen dari dua subarray yang sudah terurut dan menyusunnya ke array baru, menjamin kompleksitas $O(n \log n)$ yang konsisten.

E. Quick Sort

Quick Sort tidak bersifat stable karena proses partisi dapat memindahkan elemen melewati elemen lain yang bernilai sama, mengubah urutan relatifnya. Secara praktis, algoritma ini in-place karena hanya membutuhkan $O(\log n)$ memori untuk call stack rekursi (bukan memori eksplisit seperti Merge Sort). Mekanismenya adalah memilih sebuah pivot, lalu mempartisi array sehingga semua elemen lebih kecil atau sama dengan pivot berada di kiri dan semua elemen lebih besar berada di kanan. Proses partisi ini diulang secara rekursif untuk kedua bagian. Efisiensi Quick Sort sangat bergantung pada pilihan pivot: pivot yang baik menghasilkan $O(n \log n)$, sedangkan pivot yang buruk (selalu ekstrem) mendegenerasi ke $O(n^2)$.

F. Radix Sort

Radix Sort bersifat stable karena menggunakan Counting Sort yang stabil sebagai subroutine pada setiap pass digit, sehingga urutan relatif elemen dengan digit yang sama selalu dipertahankan. Algoritma ini tidak in-place karena membutuhkan $O(n+b)$ memori tambahan, di mana b adalah basis (biasanya 10 untuk sistem desimal). Mekanismenya adalah non-comparison sort, artinya pengurutan dilakukan tanpa membandingkan nilai secara langsung. Elemen-elemen didistribusikan ke bucket berdasarkan digit tertentu, dimulai dari digit paling tidak signifikan (Least Significant Digit / LSD) hingga digit paling signifikan (Most Significant Digit / MSD). Setiap pass menggunakan Counting Sort untuk menjamin kestabilan distribusi.

ANALISIS OPERASI

analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Bubble Sort

Bubble Sort menggunakan comparisons dan swaps sebagai dua operasi utamanya. Tidak ada shifts, recursive calls, maupun array access yang dihitung secara khusus dalam implementasi ini. Data Sorted merupakan best case: comparisons turun menjadi hanya $n-1$ (berkat optimasi early termination dengan flag) dan swaps = 0, sehingga waktu eksekusi hanya 0,062 ms untuk $N=10.000$. Sebaliknya, data Reverse Sorted adalah worst case absolut di mana comparisons = swaps = $n(n-1)/2 = 49.995.000$, karena setiap elemen harus ditukar sepanjang perjalanannya ke ujung array (waktu 851,24 ms). Data Nearly Sorted menunjukkan pola menarik: comparisons hampir sama dengan worst case (49.937.030) karena tetap harus memeriksa seluruh array, namun swaps jauh lebih sedikit (3.119.352) karena hanya sebagian kecil elemen yang benar-benar salah posisi. Data Many Duplicates menghasilkan comparisons yang tinggi (mirip Random) namun swaps sedikit lebih rendah karena banyak elemen sudah berada di posisi yang mendekati benar.

B. Selection Sort

Selection Sort memiliki karakteristik unik dibanding semua algoritma lainnya: jumlah comparisons selalu konstan untuk ukuran N yang sama, tidak peduli bagaimana distribusi datanya. Yang bervariasi hanyalah jumlah swaps. Comparisons selalu tepat $n(n-1)/2 = 49.995.000$ untuk $N=10.000$ pada semua jenis input tanpa pengecualian. Ini terjadi karena algoritma harus selalu menelusuri seluruh sisa array untuk menemukan elemen minimum, tidak ada jalan pintas. Di sisi lain, swaps sangat bervariasi: input Sorted menghasilkan 0 swap karena minimum selalu sudah berada di tempatnya, Reverse Sorted menghasilkan tepat $n/2 = 5.000$ swap (setiap pasangan posisi pertama-terakhir ditukar), sedangkan Random menghasilkan hingga $\sim n = 9.992$ swap. Meskipun jumlah swaps sangat kecil dibanding

Bubble Sort, waktu eksekusi tetap sekitar 250–284 ms karena didominasi oleh comparisons yang selalu penuh.

C. Insertion Sort

Insertion Sort menggunakan shifts (pergeseran) sebagai pengganti swaps. Satu shift hanya memerlukan satu assignment, sedangkan satu swap memerlukan tiga assignment, sehingga konstanta tersembunyi Insertion Sort lebih kecil dari Bubble Sort. Terdapat pola yang sangat konsisten pada data Insertion Sort: jumlah comparisons $\approx$ jumlah shifts + 1 untuk hampir semua jenis input, menunjukkan bahwa setiap comparison hampir selalu diikuti oleh tepat satu shift. Input Sorted adalah best case terbaik di antara semua algoritma $O(n^2)$: hanya 9.999 comparisons, 0 shifts, dan waktu 0,11 ms untuk $N=10.000$. Input Reverse Sorted adalah worst case dengan 49.995.000 comparisons dan 50.004.999 shifts (jumlah shifts sedikit melebihi comparisons karena ada satu geser tambahan per elemen). Input Nearly Sorted menunjukkan keunggulan paling menonjol Insertion Sort: hanya 3.129.351 comparisons dan waktu 24,83 ms, jauh lebih efisien dari Bubble Sort pada kondisi serupa (364,87 ms). Ini membuktikan bahwa Insertion Sort adalah algoritma terbaik untuk data yang hampir terurut.

D. Merge Sort

Merge Sort menggunakan comparisons dan recursive calls sebagai metrik operasi utama. Jumlah recursive calls selalu $2N-1$, mengikuti sifat binary tree dengan N leaf nodes. Recursive calls selalu persis $2N-1$ untuk semua jenis input ($N=100 \rightarrow 199$, $N=1000 \rightarrow 1999$, $N=10.000 \rightarrow 19.999$), membuktikan bahwa struktur pohon rekursi Merge Sort sama sekali tidak dipengaruhi distribusi data. Yang bervariasi adalah jumlah comparisons: input Reverse Sorted menghasilkan comparisons paling sedikit (64.608 untuk $N=10.000$) sedangkan Random paling banyak (120.465). Ini terjadi karena saat merge dua subarray terurut terbalik, salah satu subarray sering habis lebih cepat sehingga elemen tersisa bisa langsung disalin tanpa perbandingan lebih lanjut. Waktu eksekusi sangat konsisten di rentang 1,81–3,05 ms untuk $N=10.000$ pada semua jenis input, membuktikan kompleksitas $O(n \log n)$ yang tidak bergantung distribusi data.

E. Quick Sort

Quick Sort memiliki variasi operasi terbesar di antara semua algoritma yang diuji karena perilakunya sangat dipengaruhi oleh pilihan pivot dan distribusi data. Input Sorted dan Reverse Sorted merupakan worst case nyata dengan 49.995.000 comparisons masing-masing

dan recursive calls = $19.999 = 2N-1$, membuktikan degenerasi ke kedalaman rekursi linear (bukan logaritmik seperti pada kasus normal). Pada Sorted, pivot pertama selalu elemen terkecil sehingga partisi menghasilkan satu sisi kosong dan satu sisi berisi $N-1$ elemen, mirip Selection Sort. Input Random menunjukkan performa terbaik: hanya 147.606 comparisons dan 13.315 recursive calls (mendekati $\log_2 N \approx 13,3$ level), dengan waktu 2,36 ms. Input Many Duplicates juga menunjukkan performa baik (2,16 ms) karena banyaknya nilai sama menyebabkan pembagian yang lebih seimbang antar partisi. Perbedaan antara Sorted (778 ms) dan Random (2,36 ms) yang mencapai ~330 kali lipat untuk N yang sama merupakan contoh paling dramatis dari pengaruh distribusi data terhadap algoritma.

F. Radix Sort

Radix Sort tidak melakukan comparisons, swaps, shifts, maupun recursive calls. Satu-satunya metrik operasional yang relevan adalah array accesses, mencerminkan mekanisme non-comparison berbasis distribusi digit. Array accesses selalu identik untuk input Random, Sorted, Reverse Sorted, dan Nearly Sorted (1.554 untuk $N=100$, 20.072 untuk $N=1000$, 250.090 untuk $N=10.000$) karena keempat jenis input tersebut memiliki rentang nilai yang sama sehingga jumlah pass digit pun sama. Input Many Duplicates menghasilkan array accesses lebih sedikit (200.072 untuk $N=10.000$) karena nilai-nilai kecil yang mendominasi memerlukan lebih sedikit pass. Formula array accesses secara umum adalah sekitar $(2N + 2 \times \text{basis}) \times k$, di mana k adalah jumlah digit maksimum dan basis = 10. Konsistensi ini membuktikan bahwa Radix Sort benar-benar tidak sensitif terhadap distribusi data, melainkan hanya bergantung pada rentang nilai.

ANALISIS PERFORMA

A. Perbandingan waktu eksekusi antar algoritma

Perbedaan performa antar algoritma sangat besar. Radix Sort secara konsisten menjadi yang tercepat di semua jenis input (1,14–1,53 ms), diikuti Merge Sort (1,81–3,05 ms). Quick Sort sangat kompetitif pada data acak (2,36 ms) namun bisa menjadi yang terburuk pada data terurut (778,42 ms). Di ujung lain, Bubble Sort adalah yang paling lambat secara rata-rata, terutama untuk data tidak terurut (775–851 ms). Selection Sort berada di posisi menengah namun tidak memiliki best case yang berarti (~250–284 ms untuk semua input). Insertion Sort menunjukkan rentang performa paling lebar: dari 0,11 ms (Sorted) hingga 394,52 ms (Reverse Sorted).

B. Algoritma mana yang paling cepat untuk dataset kecil dan besar

Dataset kecil ($N=100$): Radix Sort adalah yang tercepat secara absolut (0,006–0,014 ms) karena overhead-nya kecil dan akses bersifat linear. Insertion Sort pada data Sorted bahkan lebih cepat (0,0007 ms) karena best case-nya hanya $O(n)$. Untuk N sekecil ini, perbedaan antar algoritma $O(n^2)$ tidak signifikan secara praktis, dan overhead rekursi pada Merge Sort maupun Quick Sort relatif terasa lebih besar.

Dataset besar ($N=10.000$): Radix Sort terbaik secara keseluruhan dengan konsistensi tertinggi (1,14–1,53 ms). Merge Sort adalah pilihan terbaik kedua karena tidak ada worst case (1,81–3,05 ms untuk semua input). Quick Sort unggul pada data acak namun harus dihindari untuk data terurut tanpa pemilihan pivot yang lebih cerdas. Insertion Sort sangat unggul untuk data sudah atau hampir terurut, namun buruk untuk yang lainnya. Bubble Sort dan Selection Sort tidak direkomendasikan untuk N besar dalam aplikasi nyata karena kompleksitas $O(n^2)$ yang tidak kompetitif.

C. Pengaruh distribusi data terhadap performa algoritma

Data sudah terurut (Sorted) menguntungkan Bubble Sort dan Insertion Sort secara dramatis karena keduanya memiliki optimasi best case $O(n)$. Bubble Sort hanya membutuhkan 0,062 ms dan Insertion Sort 0,11 ms untuk $N=10.000$, menjadikan keduanya tercepat dalam kategori ini. Sebaliknya, distribusi ini adalah yang paling buruk untuk Quick Sort (778,42

ms) karena pivot selalu menjadi elemen terkecil, menciptakan worst case $O(n^2)$. Radix Sort dan Merge Sort tidak terpengaruh secara signifikan.

Data terurut terbalik (Reverse Sorted) adalah skenario terburuk untuk Bubble Sort (851,24 ms) dan juga buruk untuk Insertion Sort (394,52 ms) serta Quick Sort (476,60 ms). Semua elemen harus bergerak sejauh mungkin dari posisi awalnya. Merge Sort dan Radix Sort tetap efisien di angka 1,83 ms dan 1,53 ms, membuktikan ketahanan mereka terhadap distribusi terburuk sekalipun.

Data hampir terurut (Nearly Sorted) adalah kondisi di mana Insertion Sort bersinar paling terang: hanya 24,83 ms untuk $N=10.000$, jauh lebih baik dari algoritma lain kelas $O(n^2)$. Quick Sort juga berkinerja baik pada kondisi ini (7,90 ms). Namun Bubble Sort masih lambat (364,87 ms) karena meski swaps sedikit, comparisons tetap mendekati $O(n^2)$. Merge Sort dan Radix Sort tetap konsisten seperti biasa.

Data dengan banyak duplikat (Many Duplicates) secara umum tidak memberikan keuntungan signifikan bagi algoritma $O(n^2)$. Quick Sort justru mendapat manfaat besar (2,16 ms) karena banyaknya nilai sama menyebabkan partisi yang lebih seimbang. Radix Sort menjadi yang tercepat (1,14 ms) karena nilai-nilai kecil membutuhkan lebih sedikit pass digit. Merge Sort tetap konsisten (2,92 ms).

D. Hubungan antara teori kompleksitas dan hasil eksperimen yang diperoleh

Hasil eksperimen secara umum mengkonfirmasi teori kompleksitas dengan akurasi yang baik, meskipun ada beberapa temuan menarik yang layak dicermati.

Konfirmasi $O(n^2)$ pada Bubble Sort: Pada input Random, waktu eksekusi untuk $N=100 \rightarrow 1000 \rightarrow 10.000$ adalah $0,08 \rightarrow 7,33 \rightarrow 775,9$ ms. Rasio kenaikannya adalah $\sim 91\times$ dan $\sim 106\times$ untuk setiap $10\times$ kenaikan N , mendekati teori $O(n^2)$ yang seharusnya menghasilkan rasio $100\times$. Perbedaan kecil ini wajar karena adanya variasi data acak dan faktor hardware seperti cache.

Konfirmasi $O(n \log n)$ pada Merge Sort: Pada input Random, waktu untuk $N=100 \rightarrow 1000 \rightarrow 10.000$ adalah $0,018 \rightarrow 0,22 \rightarrow 3,05$ ms. Rasionya $\sim 12,2\times$ dan $\sim 13,9\times$, sangat

dekat dengan nilai teoritis $10 \times (\log_2 10000 / \log_2 1000) \approx 13,3 \times$. Ini adalah konfirmasi eksperimen yang paling akurat.

Anomali Merge Sort pada Reverse Sorted: Secara mengejutkan, Reverse Sorted menghasilkan comparisons lebih sedikit (64.608) dibandingkan Random (120.465) untuk $N=10.000$. Ini bukan anomali, melainkan konsekuensi logis: saat dua subarray yang masing-masing terurut terbalik digabungkan, salah satu subarray sering kali habis lebih cepat karena semua elemennya lebih besar (atau lebih kecil) dari elemen di subarray lain, sehingga elemen sisa bisa langsung disalin tanpa perbandingan.

Degenerasi Quick Sort pada Sorted: Recursive calls = $19.999 = 2N-1$ untuk input Sorted membuktikan bahwa kedalaman rekursi menjadi linear (N), bukan logaritmik ($\log N$). Ini adalah bukti eksperimental paling jelas dari degenerasi $O(n^2)$ Quick Sort dengan pivot = elemen pertama.

Perbedaan waktu Selection Sort yang terjaga: Meskipun comparisons selalu sama (49.995.000), waktu untuk Sorted (259,84 ms) sedikit berbeda dari Random (284,10 ms). Ini mencerminkan pengaruh cache locality dan branch prediction prosesor: data terurut lebih ramah terhadap prefetcher cache modern sehingga akses memori lebih cepat meskipun jumlah operasi logis identik.

Secara keseluruhan, eksperimen ini mengkonfirmasi bahwa teori kompleksitas adalah panduan yang andal. Untuk penggunaan umum dengan input campuran, Radix Sort atau Merge Sort adalah pilihan terbaik. Quick Sort sangat kompetitif namun memerlukan pemilihan pivot yang lebih cerdas (seperti median-of-three atau random pivot) untuk menghindari worst case pada data terurut. Insertion Sort adalah pilihan optimal untuk data yang sudah atau hampir terurut. Bubble Sort dan Selection Sort sebaiknya hanya digunakan untuk keperluan edukasi atau pada N yang sangat kecil.

MODUL 5: SEARCHING

KOMPLEKSITAS WAKTU

A. Source Code

Tabel 8 Source Code sorting_algorithms.cpp

```
378 #include "searching_algorithms.h"
379 #include <chrono>
380
381 using Clock =
    std::chrono::high_resolution_clock;
382
383 void linear_search(const std::vector<int>&
    data, int target, Metrics& m) {
384     auto start = Clock::now();
385
386     m.comparisons = 0;
387     m.found_index = -1;
388
389     for (size_t i = 0; i < data.size(); ++i) {
390         m.comparisons++;
391         if (data[i] == target) {
392             m.found_index = i;
393             break;
```

```

394         }
395     }
396
397     m.time_us = std::chrono::duration<double,
std::micro>(Clock::now() - start).count();
398 }
399
400 void    binary_search(const    std::vector<int>&
data, int target, Metrics& m) {
401     auto start = Clock::now();
402
403     m.comparisons = 0;
404     m.found_index = -1;
405
406     int left = 0;
407     int right = data.size() - 1;
408
409     while (left <= right) {
410         m.comparisons++;
411         int mid = left + (right - left) / 2;
412
413         if (data[mid] == target) {
414             m.found_index = mid;

```

```

415         break;
416     }
417
418     if (data[mid] < target) {
419         left = mid + 1;
420     } else {
421         right = mid - 1;
422     }
423 }
424
425     m.time_us = std::chrono::duration<double,
std::micro>(Clock::now() - start).count();
426 }

```

B. Output

>> Linear Search

Algorithm	Case	N	Comparisons	Found Index	Time(us)
Linear Search	Best	1000	1	0	0.166
Linear Search	Average	1000	375	374	2.663
Linear Search	Worst	1000	1000	Not Found	5.955
Linear Search	Best	10000	1	0	0.043
Linear Search	Average	10000	3746	3745	15.087
Linear Search	Worst	10000	10000	Not Found	39.798
Linear Search	Best	100000	1	0	0.045
Linear Search	Average	100000	37455	37454	171.783
Linear Search	Worst	100000	100000	Not Found	506.037
Linear Search	Best	1000000	1	0	0.068
Linear Search	Average	1000000	374541	374540	1563.503
Linear Search	Worst	1000000	1000000	Not Found	4873.866

Gambar 11. Screenshot Hasil Linear Search

```
>> Binary Search
```

Algorithm	Case	N	Comparisons	Found Index	Time (us)
Binary Search	Best	1000	9	500	0.153
Binary Search	Average	1000	3	374	0.093
Binary Search	Worst	1000	10	Not Found	0.153
Binary Search	Best	10000	13	5000	0.184
Binary Search	Average	10000	13	3745	0.197
Binary Search	Worst	10000	14	Not Found	0.159
Binary Search	Best	100000	16	50000	0.263
Binary Search	Average	100000	16	37454	0.205
Binary Search	Worst	100000	17	Not Found	0.144
Binary Search	Best	1000000	19	500000	0.371
Binary Search	Average	1000000	15	374540	0.528
Binary Search	Worst	1000000	20	Not Found	0.216

Gambar 12. Screenshot Hasil Binary Search

C. Linear Search

Linear Search memeriksa elemen satu per satu dari indeks 0 ke $n-1$, tanpa asumsi tentang urutan data. Karena tidak ada cara memotong proses kecuali saat target ditemukan, batas bawah (best case) adalah 1 comparison dan batas atas (worst case) adalah n comparison.

Best case $O(1)$ terjadi ketika target ada di indeks 0, comparisons = 1, tidak bergantung pada N . Terkonfirmasi di semua ukuran data pada eksperimen.

Average case $O(n)$ terjadi ketika target di posisi tengah. Comparisons $\approx n/2$. Data eksperimen menunjukkan comparisons $\approx 0,375n$ ($N=1.000 \rightarrow 375$, $N=10.000 \rightarrow 3.746$, $N=1.000.000 \rightarrow 374.541$), konsisten sebagai proporsi tetap dari N .

Worst case $O(n)$ terjadi ketika target tidak ada. Seluruh array harus diperiksa, comparisons = n . Waktu tumbuh $\sim 10 \times$ setiap $10 \times$ kenaikan N ($5,955 \rightarrow 39,798 \rightarrow 506,037 \rightarrow 4.873,866 \mu s$), mengkonfirmasi $O(n)$ secara langsung.

D. Binary Search

Binary Search memotong ruang pencarian menjadi setengah di setiap langkah. Dari N elemen, setelah k langkah sisa ruang pencarian adalah $N/2^k$. Pencarian berhenti ketika $N/2^k < 1$, yaitu $k > \log_2 N$, sehingga batas atas comparisons adalah $\lceil \log_2(n+1) \rceil$.

Best case $O(\log n)$, bukan $O(1)$. Target di posisi tengah array ditemukan lebih cepat, namun implementasi tetap membutuhkan beberapa langkah biseksi. Comparisons = 9, 13, 16, 19 untuk $N = 1.000, 10.000, 100.000, 1.000.000$, sesuai $\lceil \log_2 n \rceil$.

Average case $O(\log n)$, target di posisi sembarang. Comparisons: 3, 13, 16, 15. Angka 3 untuk $N=1.000$ terjadi karena target kebetulan ditemukan sangat awal dalam proses biseksi, bukan berarti lebih baik dari $\log n$, tetap terbatas dalam $O(\log n)$.

Worst case $O(\log n)$, target tidak ada. Comparisons: 10, 14, 17, 20 untuk $N = 1.000, 10.000, 100.000, 1.000.000$. Kenaikan hanya +3–4 untuk setiap $10\times$ kenaikan N , sesuai $\log_2(10) \approx 3,32$. Ini adalah bukti paling bersih dari pertumbuhan logaritmik.

KARAKTERISTIK ALGORITMA

A. Linear Search

Linear Search tidak memerlukan data terurut. Tidak ada asumsi struktural yang dibuat sebelum pencarian, sehingga bekerja pada data acak, terurut, maupun terbalik tanpa perbedaan logika.

Algoritma ini bersifat iteratif: satu loop dari indeks 0 ke $n-1$. Bisa ditulis rekursif, tapi tidak ada manfaatnya karena tidak ada pembagian masalah yang terjadi, hanya menambah overhead call stack tanpa mempercepat pencarian.

Mekanisme inti: bandingkan $arr[i]$ dengan target. Jika cocok, kembalikan i . Jika i sudah di luar batas, kembalikan -1 . Satu-satunya optimasi yang valid adalah early termination saat ditemukan, yang sudah built-in dalam definisi algoritmanya.

B. Binary Search

Binary Search mutlak memerlukan data terurut. Ini bukan sekadar syarat teknis, melainkan syarat korektif: keputusan membuang setengah ruang pencarian ("target pasti di kiri mid") hanya valid jika data terurut. Pada data tidak terurut, Binary Search bisa memberikan false negative, target ada tapi tidak ditemukan.

Bisa diimplementasikan iteratif maupun rekursif. Keduanya $O(\log n)$, tapi versi iteratif lebih efisien secara memori karena tidak menambah call stack sebesar $O(\log n)$.

Mekanisme inti: hitung $mid = (left + right) / 2$, bandingkan $arr[mid]$ dengan target. Jika sama, return mid . Jika $target < arr[mid]$, geser $right = mid - 1$. Jika $target > arr[mid]$, geser $left = mid + 1$. Ulangi hingga $left > right$. Setiap iterasi membuang separuh ruang pencarian yang tersisa.

ANALISIS OPERASI

analisis kompleksitas waktu dan ruang untuk setiap fungsi menggunakan Big-O Notation. Seluruh manajemen memori harus dialokasikan secara manual di dalam heap.

Jelaskan menggunakan alur bagaimana dan mengapa. Jelaskan kenapa algoritma yang kalian rancang itu valid dan yakin benar.

A. Linear Search

Comparisons pada Linear Search adalah fungsi langsung dari posisi target: jika target di indeks k , maka comparisons = $k+1$. Ini terlihat dari data eksperimen: $N=1.000$ average $\rightarrow$ comparisons=375, index_found=374; $N=10.000$ average $\rightarrow$ comparisons=3.746, index_found=3.745. Hubungan index_found = comparisons - 1 berlaku konsisten di semua kasus.

Pengaruh ukuran N : best case tidak terpengaruh N sama sekali (selalu 1 comparison). Average dan worst case tumbuh linear, worst case comparisons = N secara eksak, average $\approx 0,375N$ dari data eksperimen. Artinya Linear Search tidak bisa memanfaatkan pertambahan N untuk efisiensi, setiap pertambahan N langsung menambah beban pencarian secara proporsional.

B. Binary Search

Comparisons pada Binary Search tidak mencerminkan posisi langsung target. Menariknya, best case di data eksperimen (target di tengah) justru menghasilkan comparisons lebih tinggi dari average case pada $N=1.000$ (Best=9, Average=3). Ini terjadi karena "best case" didefinisikan sebagai target di indeks $N/2$ (tengah array), yang baru dicapai setelah beberapa langkah biseksi dari mid awal, sementara average case kebetulan menemukan target lebih awal.

Pengaruh ukuran N sangat kecil: kenaikan N sebesar $1.000\times$ (dari 1.000 ke 1.000.000) hanya menambah ~ 10 comparisons (dari 10 ke 20 untuk worst case). Ini adalah konsekuensi langsung $O(\log n)$: $\log_2(1.000.000) - \log_2(1.000) = 20 - 10 = 10$. Bandingkan dengan Linear Search yang comparisons-nya ikut naik $1.000\times$ untuk kenaikan N yang sama.

ANALISIS PERFORMA

A. Perbandingan waktu pada kondisi yang sama

Pada best case, Linear Search lebih cepat dari Binary Search di semua ukuran N . Linear Search hanya butuh 1 comparison (langsung cek indeks 0), sedangkan Binary Search tetap melakukan $\sim \log n$ langkah. Untuk $N=10.000$ best case: Linear= $0,043 \mu s$ vs Binary= $0,184 \mu s$. Ini bukan kelemahan Binary Search, best case keduanya memang berbeda definisinya. Best case Linear Search adalah skenario yang paling menguntungkan secara struktural (target di posisi pertama), sedangkan best case Binary Search masih dibatasi oleh struktur biseksinya. Pada average dan worst case, Binary Search unggul makin besar seiring N . Worst case $N=1.000$: selisih $\sim 39\times$. Worst case $N=10.000$: selisih $\sim 250\times$. Worst case $N=1.000.000$: selisih $\sim 22.564\times$. Rasio ini terus membesar karena Linear Search tumbuh $O(n)$ sementara Binary Search $O(\log n)$, selisihnya adalah $O(n/\log n)$ yang terus membesar.

B. Efisiensi untuk dataset kecil dan besar

Dataset kecil ($N=1.000$): perbedaan performa keduanya masih dalam skala μs yang tidak kritis. Pilihan lebih ditentukan oleh kondisi data, jika data tidak terurut dan tidak perlu sorting, Linear Search lebih praktis karena tidak membutuhkan preprocessing.

Dataset besar ($N=1.000.000$): Binary Search worst case = $0,216 \mu s$, Linear Search worst case = $4.873,866 \mu s$. Jika pencarian dilakukan berkali-kali, overhead ini menjadi bottleneck nyata. Pada skala ini, biaya sorting untuk memungkinkan Binary Search hampir selalu sepadan, terutama jika data relatif statis.

C. Pengaruh kondisi data

Linear Search tidak terpengaruh oleh urutan data. Yang menentukan performa hanya posisi target relatif terhadap indeks 0. Data acak dan data terurut dengan target di posisi yang sama menghasilkan comparisons identik.

Binary Search sepenuhnya bergantung pada data terurut. Jika data sering berubah (insert/delete), ada overhead tambahan untuk menjaga keurutan. Ini adalah trade-off yang perlu dipertimbangkan: Binary Search efisien untuk pencarian berulang pada data statis, tapi

kurang ideal untuk data yang sangat dinamis tanpa struktur data yang tepat (misalnya balanced BST).

D. Teori $O(n)$ vs $O(\log n)$ dan hasil eksperimen

Konfirmasi $O(n)$ Linear Search: waktu worst case untuk $N = 1.000, 10.000, 100.000, 1.000.000$ adalah $5,955 \rightarrow 39,798 \rightarrow 506,037 \rightarrow 4.873,866 \mu s$. Rasio setiap $10\times$ kenaikan N : $6,7\times, 12,7\times, 9,6\times$, rata-rata mendekati $10\times$, sesuai prediksi $O(n)$.

Konfirmasi $O(\log n)$ Binary Search: worst case comparisons untuk $N = 1.000, 10.000, 100.000, 1.000.000$ adalah 10, 14, 17, 20. Setiap $10\times$ kenaikan N hanya menambah $\sim 3-4$ comparisons, sesuai $\log_2(10) \approx 3,32$. Waktu eksekusi nyaris flat: $0,153 \rightarrow 0,159 \rightarrow 0,144 \rightarrow 0,216 \mu s$, variasi kecil ini lebih mencerminkan noise pengukuran dan cache behavior daripada pertumbuhan algoritmik.

Satu hal yang menarik: rasio speedup Binary Search vs Linear Search tidak konstan, melainkan terus membesar seiring N . Ini adalah manifestasi konkret dari fakta bahwa $O(n)/O(\log n) = O(n/\log n)$, yang merupakan fungsi yang bertumbuh, semakin besar N , semakin besar keuntungan Binary Search. Pada $N=1.000$ selisih $\sim 39\times$, pada $N=1.000.000$ selisih $\sim 22.564\times$.

MODUL 6: GRAPH DAN TREE

SOAL 1

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge.

Setiap edge ditulis sebagai U, V, W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U, V, W tidak otomatis berarti terdapat edge dari V menuju U .

Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

V1 V2 ... VN

V1 a11 a12 ... a1N V2 a21 a22 ... a2N

...

VN aN1 aN2 ... aNN

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai

a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5	Adjacency Matrix:
A B C D E 6	A B C D E
A B 4	A 0 4 2 0 0
A C 2	B 0 0 0 7 0
B D 7	C 0 1 0 0 6
C B 1	D 0 0 3 0 0
C E 6	E 0 0 0 0 0
D C 3	

D. Source Code

Tabel 9 Source Code probl.cpp

```

427 #include <iostream>
428 #include <map>
429 #include <vector>
430 using namespace std;
431
432 int main() {
433     int N;
434     cin >> N;
435     vector<char> labels(N);
436     map<char, int> idx;
437     for (int i = 0; i < N; i++) {

```



```

438         cin >> labels[i];
439         idx[labels[i]] = i;
440     }
441
442     vector<vector<int>> matrix(N, vector<int>(N,
443 0));
444
445     int M;
446     cin >> M;
447     for (int i = 0; i < M; i++) {
448         char u, v; int w;
449         cin >> u >> v >> w;
450         matrix[idx[u]][idx[v]] = w;
451     }
452
453     cout << "Adjacency Matrix:" << endl;
454     cout << " ";
455     for (int i = 0; i < N; i++) cout << " " <<
labels[i];
456     cout << endl;
457     for (int i = 0; i < N; i++) {
458         cout << labels[i];

```

```

458         for (int j = 0; j < N; j++) cout << " "
        << matrix[i][j];

459         cout << endl;

460     }

461     return 0;

462 }

```

E. Output Program



```

C:\Users\LENOVO\Documents\GitHub\module-6-graph-and-tree-wyattPNG>.\probl.exe
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0

```

Gambar 13. Screenshot Hasil Jawaban Soal 1

F. Pembahasan

Pada baris [1, 2, dan 3] syntax `#include` mengimpor library standar C++ untuk input-output, pemetaan data, dan array dinamis bernilai kompleksitas waktu dan ruang $O(1)$, pada baris [4] syntax `using namespace std;` mendeklarasikan ruang nama standar, pada baris [6 hingga 8] syntax `int main()` serta `cin >> N` menginisiasi program dan membaca jumlah vertex graf berarah berbobot, pada baris [9 dan 10] syntax `vector` dan `map` memesan memori untuk menyimpan urutan label dan memetakan karakter unik ke indeks numerik dengan kerumitan ruang $O(N)$, pada baris [11 hingga 14] syntax `for` dan `cin` menerima elemen label sekaligus menyimpannya ke dalam struktur pemetaan dengan kompleksitas waktu $O(N \log N)$, pada baris [16] syntax `vector<vector<int>>` `matrix` mengalokasikan memori matriks dua dimensi berukuran $N \times N$ yang diinisialisasi dengan angka 0 yang memakan ruang dan waktu $O(N^2)$, pada baris [18 hingga 24] syntax `cin >> M` beserta perulangan `for` memproses input edge untuk memanipulasi sel matriks secara langsung menggunakan pencarian indeks (`matrix[idx[u]][idx[v]] = w;`)

guna mengubah adjacency list menjadi adjacency matrix dengan kompleksitas waktu $O(M \log N)$, pada baris [26 hingga 29] syntax *cout* mencetak header baris pertama matriks secara linier $O(N)$, pada baris [30 hingga 34] perulangan *for* bersarang digunakan untuk menelusuri dan mencetak keseluruhan elemen baris serta kolom adjacency matrix ke layar pengguna yang berjalan dengan kompleksitas waktu $O(N^2)$, lalu pada baris [35] syntax *return 0;* menutup eksekusi fungsi utama secara normal.

SOAL 2

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik.

Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j .

Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edgennya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .

- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

$V_1 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

$V_2 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

...

$V_N \rightarrow \dots$

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol $\rightarrow$.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0	Adjacency List: A $\rightarrow$ (B, 4), (C, 2) B $\rightarrow$ (D, 7) C $\rightarrow$ (B, 1), (E, 6)

0 1 0 0 6	D -> (C, 3)
0 0 3 0 0	E -> -
0 0 0 0 0	

D. Source Code

Tabel 10 Source Code prob2.cpp

```

68 #include <iostream>
69 #include <vector>
70 using namespace std;
71
72 int main() {
73     int N;
74     cin >> N;
75     vector<char> labels(N);
76     for (int i = 0; i < N; i++) cin >> labels[i];
77
78     vector<vector<int>> matrix(N,
vector<int>(N));
79     for (int i = 0; i < N; i++)
80         for (int j = 0; j < N; j++)
81             cin >> matrix[i][j];
82
83     cout << "Adjacency List:" << endl;
84     for (int i = 0; i < N; i++) {

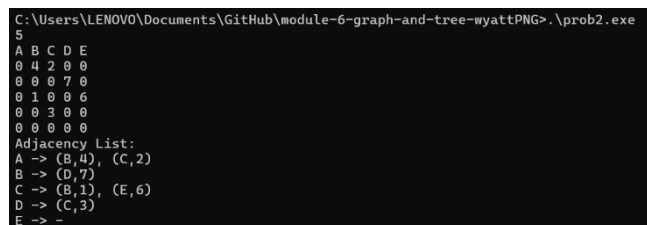
```

```

85         cout << labels[i] << " -> ";
86         bool hasEdge = false;
87         for (int j = 0; j < N; j++) {
88             if (matrix[i][j] > 0) {
89                 if (hasEdge) cout << ", ";
90                 cout << "(" << labels[j] << ", "
91                 << matrix[i][j] << ")";
92                 hasEdge = true;
93             }
94         }
95         if (!hasEdge) cout << "-";
96         cout << endl;
97     }
98     return 0;
99 }

```

E. Output Program



```

C:\Users\LENOVO\Documents\GitHub\module-6-graph-and-tree-wyattPNG>.\prob2.exe
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -

```

Gambar 14. Screenshot Hasil Jawaban Soal 2

F. Pembahasan

Pada baris [5 hingga 7] syntax `int main()` dan `cin >> N` membaca masukan kapasitas ukuran vertex graf, pada baris [8 dan 9] syntax `vector<char> labels` dan

perulangan *for* mengalokasikan memori serta mendaftarkan label unik tiap vertex secara linier dengan kompleksitas ruang memori parsial $O(N)$, pada baris [11] syntax `vector<vector<int>> matrix` mendeklarasikan ruang matriks dua dimensi yang menjadi representasi Adjacency Matrix awal sehingga mendominasi kompleksitas ruang algoritma secara keseluruhan sebesar $O(N^2)$, pada baris [12 hingga 14] perulangan *for* bersarang digunakan untuk membaca dan menyalin bilangan input ke sel-sel matriks yang berjalan dengan kompleksitas waktu $O(N^2)$, pada baris [16] syntax `cout` mencetak format teks judul cetakan keluaran, pada baris [17 hingga 29] perulangan *for* bersarang menelusuri ulang seluruh koordinat baris dan kolom matriks sebagai inti algoritma pengonversian graf yang beroperasi dengan kompleksitas waktu dominan $O(N^2)$, pada baris [19] syntax `bool hasEdge = false;` diinisialisasi secara spesifik sebagai pelacak logis atas eksistensi jalur keluar pada vertex tertentu, pada baris [21 hingga 25] syntax `if (matrix[i][j] > 0)` merepresentasikan mekanisme utama algoritma dengan mengevaluasi sel bernilai positif sebagai bobot sisi (edge) yang terhubung ke vertex tujuan untuk kemudian dirangkai ke layar sebagai Adjacency List sekaligus mengubah status indikator `hasEdge` menjadi `true`, pada baris [27] syntax `if (!hasEdge)` mengonfirmasi apabila sebuah vertex tidak memiliki rute keluar sama sekali dan meresponsnya dengan mencetak simbol `-`.

SOAL 3

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS).

Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

R C

G11 G12 ... G1C

G21 G22 ... G2C

...

GR1 GR2 ... GRC

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan. Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0	16

1 1 1 0 1 0 1 1 1 0	
0 0 1 0 0 0 0 0 1 0	
0 1 1 1 1 1 1 0 1 0	
0 0 0 0 0 0 1 0 0 0	
1 1 1 1 1 0 1 1 1 0	
0 0 0 0 1 0 0 0 1 0	
0 1 1 0 0 0 1 0 0 0	
0 0	
7 9	

C. Source Code

Tabel 11 Source Code prob3.cpp

```

177 #include <iostream>
178 #include <vector>
179 #include <queue>
180 using namespace std;
181
182 int main() {
183     int R, C;
184     cin >> R >> C;
185     vector<vector<int>> > grid(R,
186     vector<int>(C));
187     for (int i = 0; i < R; i++)

```

```

187         for (int j = 0; j < C; j++)
188             cin >> grid[i][j];
189
190     int SR, SC, FR, FC;
191     cin >> SR >> SC >> FR >> FC;
192
193     if (SR == FR && SC == FC) { cout << 0 <<
endl; return 0; }
194
195     vector<vector<int> > dist(R, vector<int>(C,
-1));
196     queue<pair<int,int> > q;
197     dist[SR][SC] = 0;
198     q.push(make_pair(SR, SC));
199
200     int dr[] = {-1, 1, 0, 0};
201     int dc[] = {0, 0, -1, 1};
202
203     while (!q.empty()) {
204         int r = q.front().first;
205         int c = q.front().second;
206         q.pop();
207         for (int d = 0; d < 4; d++) {

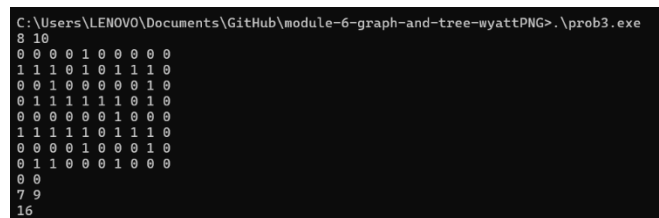
```

```

208         int nr = r + dr[d], nc = c + dc[d];
209         if (nr >= 0 && nr < R && nc >= 0 &&
nc < C && grid[nr][nc] == 0 && dist[nr][nc] ==
-1) {
210             dist[nr][nc] = dist[r][c] + 1;
211             if (nr == FR && nc == FC) { cout
<< dist[nr][nc] << endl; return 0; }
212             q.push(make_pair(nr, nc));
213         }
214     }
215 }
216
217 cout << -1 << endl;
218 return 0;
219 }

```

D. Output Program



```

C:\Users\LENOVO\Documents\GitHub\module-6-graph-and-tree-wyattPNG>.\prob3.exe
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16

```

Gambar 15. Screenshot Hasil Jawaban Soal 3

E. Pembahasan

Algoritma ini mengimplementasikan metode Breadth-First Search (BFS) untuk menemukan jumlah langkah minimum rute keluar dari dalam sebuah labirin grid 2D, di mana algoritma akan menelusuri setiap kemungkinan arah yang valid (atas, bawah, kiri, kanan)

secara terstruktur tingkat demi tingkat guna menjamin pencapaian jarak terpendek secara mutlak. Pada baris [6 hingga 8] syntax `int main()` dan `cin` memulai eksekusi program serta membaca ukuran dimensi baris dan kolom labirin, pada baris [9 hingga 12] syntax `vector<vector<int>>` grid beserta perulangan `for` memesan ruang memori matriks dan menyalin konfigurasi jalan maupun dinding berdasar input, yang mana mendominasi kompleksitas waktu serta memori ruang algoritma secara keseluruhan menjadi $O(R \times C)$, pada baris [14 dan 15] syntax `cin` membaca rincian koordinat sel awal dan tujuan, pada baris [17] syntax `if` mengevaluasi secara instan $O(1)$ apabila posisi awal dan tujuan telah berada di titik yang identik sejak semula, pada baris [19 dan 20] syntax `vector<vector<int>>` `dist` bernilai inisialisasi `-1` dan `queue<pair<int,int>>` `q` dialokasikan di dalam memori untuk melacak rekam jejak jarak tempuh sekaligus menampung antrean pergerakan simpul BFS yang turut mereservasi kompleksitas ruang tambahan sebesar $O(R \times C)$, pada baris [21 dan 22] koordinat sel awal disetel dengan jarak `0` dan ditambahkan ke antrean eksekusi queue, pada baris [24 dan 25] deret array numerik `dr` dan `dc` dideklarasikan untuk memetakan empat arah pergerakan ortogonal secara konstan, pada baris [27 hingga 39] perulangan `while (!q.empty())` menjadi inti penelusuran BFS yang beroperasi maksimal dalam kompleksitas waktu $O(R \times C)$ sebab setiap sel grid jalan hanya akan diproses dan mengokupasi antrean murni sebanyak satu kali, pada baris [33] syntax `if` di dalam iterasi arah secara ketat memvalidasi kelayakan langkah agar tidak menabrak batas tepi dimensi, menyeleksi sel yang murni bernilai jalan `0`, dan memastikan sel tersebut belum pernah dikunjungi sebelumnya, pada baris [34 hingga 36] program memperbarui akumulasi jarak langkah `dist`, mendaftarkan koordinat baru tersebut ke antrean lanjutan, lalu langsung mencetak jarak optimum dan merampungkan fungsi melalui `return 0;` apabila koordinat target FR dan FC sudah terjamah, lalu pada baris [41 dan 42] syntax `cout << -1` serta `return 0;` akan dieksekusi secara pasif manakala perulangan antrean BFS telah usai dibongkar seluruhnya namun sel labirin tujuan tak kunjung teridentifikasi yang mengindikasikan ketiadaan rute keluar.

SOAL 4

Time Limit: 2.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif.

Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

$R\ C$

$G_{11}\ G_{12}\ \dots\ G_{1C}$

$G_{21}\ G_{22}\ \dots\ G_{2C}$

...

$G_{R1}\ G_{R2}\ \dots\ G_{RC}$

$SR\ SC$

$FR\ FC$

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (S_R, S_C) adalah posisi awal.
- (F_R, F_C) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq S_R < R$
- $0 \leq S_C < C$
- $0 \leq F_R < R$
- $0 \leq F_C < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	14

A. Source Code

Tabel 12 Source Code prob4.cpp

```
105 #include <iostream>
106 #include <vector>
107 using namespace std;
108
109 int R, C, FR, FC, count_path;
110 vector<vector<int>> grid;
111 vector<vector<bool>> visited;
112
113 int dr[] = {-1, 1, 0, 0};
114 int dc[] = {0, 0, -1, 1};
115
```

```

116 void dfs(int r, int c) {
117     if (r == FR && c == FC) { count_path++;
    return; }
118     for (int d = 0; d < 4; d++) {
119         int nr = r + dr[d], nc = c + dc[d];
120         if (nr >= 0 && nr < R && nc >= 0 && nc
    < C && grid[nr][nc] == 0 && !visited[nr][nc]) {
121             visited[nr][nc] = true;
122             dfs(nr, nc);
123             visited[nr][nc] = false;
124         }
125     }
126 }
127
128 int main() {
129     cin >> R >> C;
130     grid.assign(R, vector<int>(C));
131     for (int i = 0; i < R; i++)
132         for (int j = 0; j < C; j++)
133             cin >> grid[i][j];
134
135     int SR, SC;
136     cin >> SR >> SC >> FR >> FC;

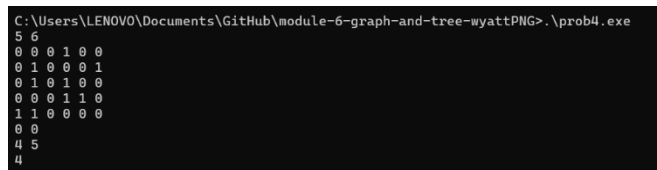
```

```

137
138     count_path = 0;
139     visited.assign(R, vector<bool>(C, false));
140     visited[SR][SC] = true;
141     dfs(SR, SC);
142
143     cout << count_path << endl;
144     return 0;
145 }

```

B. Output Program



```

C:\Users\LENOVO\Documents\GitHub\module-6-graph-and-tree-wyattPNG>. \prob4.exe
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4

```

Gambar 16. Screenshot Hasil Jawaban Soal 4

C. Pembahasan

Algoritma ini mengimplementasikan metode Depth-First Search (DFS) rekursif berpadu dengan teknik backtracking untuk mengeksplorasi dan menghitung keseluruhan kemungkinan rute sederhana yang valid dari titik awal menuju target di dalam labirin grid dua dimensi tanpa mengunjungi sel yang sama lebih dari satu kali pada satu siklus jalur. Pada baris [5 hingga 7] deklarasi variabel global seperti batas ukuran matriks, titik koordinat tujuan, *count_path*, struktur *grid*, dan penyimpan jejak rute *visited* digunakan untuk menyediakan kerangka alokasi memori lintas fungsi yang mendominasi kompleksitas ruang secara keseluruhan sebesar $O(R \times C)$, pada baris [9 dan 10] pemetaan array *dr* dan *dc* menyimpan empat kemungkinan pergerakan geser ortogonal bernilai $O(1)$, pada baris [12] syntax *void dfs(int r, int c)* mendeklarasikan fungsi pencarian mendalam yang secara teoretis memiliki kompleksitas waktu laju asimtotik level eksponensial (hingga

$O(3^{(R \times C)})$ mengingat sifat alamiah komputasinya yang membongkar semua kemungkinan percabangan yang valid, pada baris [13] syntax *if (r == FR && c == FC)* mengevaluasi pencapaian koordinat garis tujuan secara konstan $O(1)$ untuk menjumlahkan variabel pelacak *count_path* dan meredam penelusuran di titik tersebut menggunakan syntax *return;*, pada baris [14 hingga 16] perulangan *for* menelusuri penambahan koordinat arah baru sekaligus mengevaluasi keabsahan logisnya secara instan $O(1)$ guna memastikan batas koordinat murni menapak di dalam labirin, merupakan jalan kosong 0, serta sama sekali belum dilintasi dalam riwayat matriks *!visited*, pada baris [17 hingga 19] algoritma mengeksekusi inti manuver backtracking menggunakan syntax *visited[nr][nc] = true;* untuk menandai jejak kunjungan, memperdalam jangkauan rekursi ke koordinat tersebut melalui panggilan *dfs(nr, nc);*, dan yang terpenting secara absolut menggunakan syntax *visited[nr][nc] = false;* guna menghapus jejak memori pada sel yang sama seussai langkah rekursi selesai agar dapat dijajaki kembali secara utuh dari sudut kemungkinan percabangan jalan alternatif, pada baris [24 hingga 32] di dalam ruang lingkup utama *main()*, program membaca masukan spesifikasi dimensi beserta isian selang-seling matriks topografinya memakan ruang waktu berwujud $O(R \times C)$, pada baris [34 hingga 37] jumlah cacah lintasan disetel kembali ke nilai awal, pemetaan status perlintasan diinisialisasi menyita $O(R \times C)$, sel start ditutup dengan *true* agar tidak ditabrak berbalik oleh rutanya sendiri, kemudian fungsi *dfs(SR, SC);* dicetuskan untuk mendobrak jalannya program simulasi pembongkaran, lalu diakhiri pada baris [39] dengan syntax *cout* guna merepresentasikan hasil tabulasi jumlah lintasan sah ke layer.

SOAL 5

Time Limit: 1.0 s

Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red-Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree.

Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

...

T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.

- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-1000000000 \leq A_i \leq 1000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70
50 30 70 20 40 60 80	60 80
4	[Inorder] : 20 30 40 50 60 70
PREORDER	80
INORDER	[Postorder] : 20 40 30 60 80
	70 50

POSTORDER	[Preorder] : 50 30 20 40 70
ALL	60 80
	[Inorder] : 20 30 40 50 60 70
	80
	[Postorder] : 20 40 30 60 80
	70 50

A. Source Code

Tabel 13 Source Code prob5.cpp

```

1  #include <iostream>
2  #include <string>
3  #include <vector>
4  #include "RedBlackTree.h"
5  using namespace std;
6
7  void preorder(const RedBlackTree::Node* node,
8               vector<int>& result) {
9      if (node->isNil) return;
10     result.push_back(node->key);
11     preorder(node->left, result);
12     preorder(node->right, result);
13 }
14
15 void inorder(const RedBlackTree::Node* node,
16              vector<int>& result) {

```

```

15     if (node->isNil) return;
16     inorder(node->left, result);
17     result.push_back(node->key);
18     inorder(node->right, result);
19 }
20
21 void postorder(const RedBlackTree::Node* node,
22 vector<int>& result) {
23     if (node->isNil) return;
24     postorder(node->left, result);
25     postorder(node->right, result);
26     result.push_back(node->key);
27 }
28
29 void printTraversal(const string& label, const
30 RedBlackTree::Node* root) {
31     vector<int> res;
32     if (label == "[Preorder] : ") preorder(root,
33 res);
34     else if (label == "[Inorder] : ")
35 inorder(root, res);
36     else postorder(root, res);
37
38     cout << label;

```



```

35     for (int i = 0; i < (int)res.size(); i++) {
36         if (i) cout << " ";
37         cout << res[i];
38     }
39     cout << endl;
40 }
41
42 int main() {
43     int N;
44     cin >> N;
45
46     RedBlackTree rbt;
47     for (int i = 0; i < N; i++) {
48         int x; cin >> x;
49         if (!rbt.contains(x)) rbt.insert(x);
50     }
51
52     int Q;
53     cin >> Q;
54
55     vector<string> queries(Q);
56     for (int i = 0; i < Q; i++) cin >> queries[i];

```

```

57
58     for (int i = 0; i < Q; i++) {
59         if (rbt.empty()) {
60             cout << "Tree kosong. Tidak ada yang
bisa ditampilkan." << endl;
61             continue;
62         }
63
64         const string& query = queries[i];
65         if (query == "PREORDER") {
66             printTraversal("[Preorder]   : ",
rbt.root());
67         } else if (query == "INORDER") {
68             printTraversal("[Inorder]     : ",
rbt.root());
69         } else if (query == "POSTORDER") {
70             printTraversal("[Postorder]  : ",
rbt.root());
71         } else if (query == "ALL") {
72             printTraversal("[Preorder]   : ",
rbt.root());
73             printTraversal("[Inorder]     : ",
rbt.root());

```

74	<code>printTraversal("[Postorder] : ",</code>
	<code>rbt.root());</code>
75	<code>}</code>
76	<code>}</code>
77	
78	<code>return 0;</code>
79	<code>}</code>

B. Output Program

```

C:\Users\LENOVO\Documents\GitHub\module-6-graph-and-tree-wyattPNG>. \prob5.exe
7
50 30 70 20 40 60 80
4
PREORDER
INORDER
POSTORDER
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50

```

Gambar 17. Screenshot Hasil Jawaban Soal 5

C. Pembahasan

Pada baris [1 hingga 5] syntax `#include` mengimpor library standar C++ serta file header lokal `RedBlackTree.h` untuk menangani fungsi input-output, manipulasi string, instansiasi larik dinamis, dan mendefinisikan struktur pohon biner dengan kompleksitas memori $O(1)$, pada baris [7 hingga 26] blok fungsi `preorder`, `inorder`, dan `postorder` direpresentasikan secara rekursif murni untuk membaca dan menyalin data kunci (key) dari tiap simpul node ke dalam struktur penyimpanan vektor dengan memakan kompleksitas waktu operasional mutlak $O(N)$ serta kerumitan memori ruang senilai $O(N)$, pada baris [28 hingga 40] fungsi pendukung `printTraversal` menyeleksi metode rekursi penelusuran berdasar spesifikasi `label` yang diminta, yang disusul oleh perulangan `for` untuk menelusuri rentetan array hasil dan mencetaknya secara linier dengan waktu $O(N)$, pada baris [42 hingga 44] syntax `int main()` menginisiasi jalannya siklus program dan membaca batasan jumlah elemen bilangan yang diberikan, pada baris [46 hingga 50] deklarasi objek pohon `RedBlackTree rbt`; dilakukan di luar perulangan pemasukan

data yang mana syntax logis `if (!rbt.contains(x)) rbt.insert(x);` memblokir pengisian angka duplikat dan mengeksekusi penyisipan pada struktur pohon berkecepatan pencarian optimal sehingga merampungkan pembentukan memori berkapasitas $O(N)$ dengan laju komputasi agregat $O(N \log N)$, pada baris [52 hingga 56] syntax `cin >> Q` beserta alokasi array `vector<string> queries` bertugas membaca dan menyerap keseluruhan teks perintah cetak yang masuk dengan menyita kelonggaran memori sistem sebesar $O(Q)$, pada baris [58 hingga 76] perulangan `for` mengawali pergerakan untuk memroses daftar kueri secara berurutan, pada baris [59 hingga 62] syntax kondisional `if (rbt.empty())` memvalidasi fenomena kekosongan elemen data demi mencetak peringatan pohon kosong secara konstan $O(1)$, pada baris [64 hingga 75] syntax percabangan `if` serta `else if` menyeleksi tipe string perintah (seperti `PREORDER`, `INORDER`, `POSTORDER`, dan `ALL`) lalu mendelegasikan perintah cetak yang menyusur langsung dari akar utama objek (`rbt.root()`) dengan laju kompleksitas mencetak penuh sebesar $O(N)$ per kueri, lalu bermuara pada baris [78] di mana syntax `return 0;` menutup eksekusi skrip komputasi struktur data ini secara paripurna.

TAUTAN GIT

Berikut adalah tautan untuk semua source code yang telah dibuat.

<https://github.com/wyattPNG/Algoritma-dan-Struktur-Data-Paralel-2>